

所属类别	2025 年“华数杯”全国大学生数学建模竞赛	参赛编号
本科组		CM2502665

基于混合整数非线性规划的 5G 切片资源管理问题研究

摘要

5G 基站作为目前维持移动通信的重要设施，采用两层的异构网络和网络切片技术提升不同用户的服务质量。本文通过建立混合整数非线性优化模型，分析了实现用户服务质量最优的资源块分配、基站的发射功率分配和用户接入基站分配决策。

针对问题一，我们以最大化整体的用户服务质量为目标函数，以资源块可覆盖的用户数作为决策变量，以最大化服务质量为目标函数，建立**整数优化模型**。因为本题数据量较少，所以使用**分析缩域的穷举搜索**求解，相较于一般的穷举搜索，搜索范围更小，节约算力和时间。本题求解结果为当分配给 **URLLC20 个资源块，eMBB20 个资源块，mMTC10 个资源块**时，服务质量最大且符合三种切片的特点，服务质量为 12.2806。

针对问题二，本文将 1000ms 的动态用户任务分为十次 100ms 的决策，根据编号靠前的用户优先处理的原则，考虑排队导致的排队延迟。于是我们以最大化服务质量为目标函数，以各决策的资源块可覆盖的用户数作为决策变量，建立**混合整数非线性优化模型**。因为存在积压在排队队列中的用户任务，本题使用按**贪心算法**思想，用**滚动优化算法**寻找每个决策时刻 t 的局部最优解，从而得到全局次优解。

针对问题三，本文先假设任务选择传输延迟最小的基站，进行整体的干扰程度最小且各基站的任务量均衡的**多目标优化**。我们发现**功率与干扰程度、任务量的一般规律**：当功率分配趋向于两个极小功率，一个极大功率时整体干扰最小，三个功率趋于相同时，干扰最大；功率较大的基站任务量较大。我们以最大化服务质量为目标函数，以各决策的资源块可覆盖的用户数和各基站功率作为决策变量，建立**混合整数非线性优化模型**。由于模型复杂，本文使用**分层迭代优化算法**求解。

针对问题四，根据问题三发现的规律，设置接入基站分配、资源块分配、发射功率控制三种决策变量，以所有用户累积服务质量最大为目标函数建立**混合整数非线性优化模型**，注意**宏基站不受干扰**。由于变量维度高，直接求解难度大，本文使用**分层迭代优化算法**求解。

针对问题五，本文同样设置三种决策变量，把问题四的最优服务质量作为约束条件，以能耗最小化为目标函数，建立**混合整数非线性优化模型**，将题目中的能耗、质量**双目标优化问题转化为单目标优化**。我们按照**枚举法**的思路对模型进行求解。

关键字： 5G 多目标优化 混合整数非线性优化 分层迭代优化算法

一、问题重述

1.1 问题一重述

一个微基站拥有 50 个资源块用于分配。当下每个用户都只到达了一个任务，将所有的资源块分配给三类切片，用于响应用户的下行传输需求。求出具体的资源块分配方案，使用户服务质量达到最大。

1.2 问题二重述

现有用户会在 1000 ms 内以概率进行多次传输请求，且用户会进行移动，导致信道发生变化，因此系统在每隔一段时间对资源重新进行分配。中间需要对资源进行 10 次决策，分配时要同时考虑即将到达的任务和积压在排队队列中的用户任务，求出每次决策的三类切片的资源块最佳分配方案，使得整体的用户服务质量达到最大。

1.3 问题三重述

针对多个微基站的情况，不同的传输链路之间可能存在干扰。基站在对资源块进行分配的同时，还需要对每个基站的发射功率进行控制，以降低系统整体的干扰程度。功率决策决定了每个切片内所有资源块统一的功率，基站功率的取值范围为 $p \in [10, 30]dBm$ 。求出各个基站每次决策的资源块分配方案和功率控制方案，使系统的用户服务质量达到最大。

1.4 问题四重述

宏基站和微基站两类基站之间不存在干扰，仅微基站之间存在干扰。需要确定每个用户的接入决策，即是由最近的微基站服务还是由宏基站服务。假定宏基站持有 100 个资源块，功率范围为 $\in [10, 40]dBm$ ，和周围多个拥有 50 个资源块的微基站共同服务周边的用户，针对所有用户的任务需求，求出每个用户的接入决策，每个基站的切片划分决策以及基站的发射功率分配，以达到最大用户服务质量。

1.5 问题五重述

在问题的模型中，针对用户的任务情况以最大用户服务质量进行了资源调度，根据能耗计算模型和问题四的数据给出合理的资源分配策略，使得能耗最低的同时能够达到最大的用户服务质量。

二、模型假设

- (1) 假设系统每 100 ms 决策一次系统的资源配置方案。
- (2) 假设资源块在频域上的跨度为 360kHz，在时域上的跨度为 1ms。
- (3) 为了不造成资源浪费，假设每个用户仅能分配到其切片规定数目的资源块。
- (4) 因为单个资源块是不可分割的最小资源单位，假设每个用户占用资源块时间为单个资源块时域的整数倍。
- (5) 假设一个任务只能由一个基站完成，不可由多个基站合作完成。

三、符号说明

表 1 主要符号说明

符号	意义	单位
$\phi_{n,k}$	用户 k 与基站 n 之间的大规模衰减	dB
$h_{n,k}$	小规模瑞利衰减	/
D	任务数据量	Mbit
b	每个资源块带宽	Hz
$p_{n,k}$	发射功率	dBm
r	传输速率	Mbps
i	资源块块数	/
p_{rx}	接受功率	mW
N_0	白噪声	mW 或 dBm
t	决策时刻	ms
Qos	用户服务质量	/
γ	信干噪比	/

四、问题分析

4.1 问题一分析

假设本题的微基站发射功率为 30dBm，根据附件 1 中 $\phi_{n,k}$ 和 $h_{n,k}$ 信息结合附录内容，经过数据计算和处理（单位换算）得到各用户的下行传输速率。通过各用户任务量和下行传输速率可求得各用户任务传输延迟 T。附录中网络切片参数表得到 URLLC、eMBB、mMTC 三种切片的每个用户的资源块占用量、最低速率、最大延迟和惩罚系数。因为仅有 50 个资源块（资源块不足以一次完成所有任务），存在 Q 排队时延，所以要**规避三种切片的惩罚**，优化资源块分配使用户服务质量总和最大，建立单任务各用户资源块分配优化模型。本题仅一个基站，不考虑干扰信号。

4.2 问题二分析

假设本题的微基站发射功率仍为 30dBm，根据附件 2 中 $\phi_{n,k}$ 和 $h_{n,k}$ 信息结合附录内容，经过数据计算和处理（单位换算）得到各用户每个任务的传输速率。通过各用户任务量和下行传输速率可求得各用户任务传输延迟 T。本题要求分配时不仅要考虑即将到达的任务，还需要考虑积压在排队队列中的用户任务。因此要考虑排队的情形，依照附录中的要求，**优先处理编号靠前的用户**。本题要求中间需要对资源进行 10 次决策，实现 10 次决策的各用户服务质量总和最大，建立**动态累计**任务各切片资源块分配优化模型。本题仅一个基站，不考虑干扰信号。

4.3 问题三分析

本题给定三个微基站的发射功率范围是 [10,30]dBm，各有资源块 50 个。本题要求对每个基站的发射功率进行控制，以降低系统整体的干扰程度。根据附件 3 所给数据和附录中的信号干扰模型，我们发现发射功率不仅影响干扰程度，而且影响基站传输速率。假设任务由传输时延最小的基站接入，进行整体干扰程度最低且三个基站的任务量均衡的**多目标优化**。本题还要求通过资源块分配和功率控制，使系统的用户服务质量达到最大。于是把多目标优化结论应用到约束中，中间需要对资源进行 10 次决策，建立多个微基站资源块分配和功率控制优化模型。

4.4 问题四分析

本题给定三个微基站的发射功率范围是 [10,30]dBm，各有资源块 50 个，一个宏基站发射功率范围是 [10,40]dBm，有 100 个资源块。本题要求考虑**每个用户的接入决策、每个基站的切片划分决策和基站的发射功率分配决策**，三种决策优化使用户服务质量最大。用户按**最小传输时延**选择是由宏基站还是最近微基站接入，这会使服务质量增加，

但也可能导致某基站任务量过大,降低服务质量,因此要**按资源块数量平衡各基站任务量**。每个基站的切片划分决策和基站的发射功率分配决策,可参考问题三的结论,但要注意基站数量和资源块总数的变化。

4.5 问题五分析

本题要求通过调节功率来降低能耗,同时保证服务质量最大。基站的能耗包括固定能耗, RB 激活能耗以及发射能耗三个部分。固定能耗不可减少; RB 激活能耗与 1000ms 使用的资源块总数有关,其随传输延迟增大而增大,应使总传输延迟最小,对于单基站来说发射功率越高,传输延迟越小, RB 激活能耗越小;发射能耗与发射功率线性正相关。本文分析了发射功率改变导致 **RB 激活能耗与发射能耗之和应有极小值**。对于服务质量最大问题,可以将问题四的结论作为约束条件来建立**能耗和质量双目标优化模型**。

五、问题建立与求解

5.1 问题一: 单任务各用户资源块分配优化模型

5.1.1 模型建立

(1) 数据处理和关键参数计算

根据附录第一节信号传输模型可以总结以下的信号传输关键参数与发射功率 $p_{n,k}$ 及两种衰减 $\phi_{n,k}$ 、 $h_{n,k}$ 等已知参数的关系式。注意 N_0 、 p_{rx} 必须统一量纲,因为信干噪比是无量纲的。

$$\left\{ \begin{array}{l} \text{接收信号 } p_{rx} \text{ 功率与发射功率及两种衰减的关系式:} \\ p_{rx}(mW) = 10^{\frac{p_{n,k}(dBm) - \phi_{n,k}(dB)}{10}} \cdot |h_{n,k}|^2 \\ \text{白噪声 } N_0 \text{ 与总带宽 (ib) 和噪声系数关系式:} \\ N_0(dBm) = -174 + 10 \log_{10}(ib) + NF \\ \text{白噪声 } N_0 \text{ 单位换算式:} \\ N_0(mW) = 10^{\frac{N_0(dBm)}{10}} \end{array} \right. \quad (1)$$

$$\text{其中} \left\{ \begin{array}{l} p_{n,k} = 30dBm \\ \phi_{n,k} \text{ 和 } h_{n,k} \text{ 数据源自附件} \\ i = \begin{cases} 10, \text{URLLC 切片} \\ 5, \text{eMBB 切片} \\ 2, \text{mMTC 切片} \end{cases} \\ b = 3.6 \times 10^5 Hz \end{array} \right. \quad (2)$$

因此此问仅有一个微基站，没有干扰信号，信干噪比（ γ ）计算式为：

$$\gamma = \frac{p_{rx}(\text{mW})}{N_0(\text{mW})} \quad (3)$$

故传输速率与总带宽和信干噪比的关系式为：

$$r(\text{Mbps}) = \frac{ib \cdot \log_2(1 + \gamma)}{10^6} \quad (4)$$

用户任务的传输延迟 T ，即传输延迟时间 = 任务量/传输速率，其计算式为：

$$T(s) = \frac{D}{r} \quad (5)$$

本题的用户任务的排队延迟 Q ，为大于或等于上一用户的传输延迟 T 的最小整数。

$$Q = \min(x), x \geq T \text{ 且 } x \in Z \quad (6)$$

总延迟由排队延迟和传输延迟组成， $L = Q + T$ 。

(2) 决策变量

本题所求具体的资源块分配方案，就是对三种切片的资源块分配决策。所以将所求结果转化为三个决策变量（资源块可覆盖的用户数）。

- x_u ：分配给 *URLLC* 切片的资源块可覆盖的用户数
- x_e ：分配给 *eMBB* 切片的资源块可覆盖的用户数
- x_m ：分配给 *mMTC* 切片的资源块可覆盖的用户数

(3) 用户服务质量 QoS 定义

根据附录第三节用户服务质量定义中给出的用户服务质量函数 (7)，可以用于推导本模型目标函数。

$$\left\{ \begin{array}{l} \textbf{URLLC 用户服务质量:} \\ y^{URLLC} = \begin{cases} \alpha^L, & L \leq L_{SLA} \\ -M^{URLLC}, & L > L_{SLA} \end{cases} \\ \textbf{eMBB 用户服务质量:} \\ y^{eMBB} = \begin{cases} 1, & r \geq r_{SLA} \text{ 且 } L \leq L_{SLA} \\ r/r_{SLA}, & r < r_{SLA} \text{ 且 } L \leq L_{SLA} \\ -M^{eMBB}, & L > L_{SLA} \end{cases} \\ \textbf{mMTC 用户服务质量:} \\ y^{mMTC} = \begin{cases} \frac{\sum_{i \in \mathcal{J}} c_i^j}{\sum_{i \in \mathcal{J}} c_i}, & L \leq L_{SLA} \\ -M^{mMTC}, & L > L_{SLA} \end{cases} \end{array} \right. \quad (7)$$

其中各参数见下表：

类别\指标	URLLC	eMBB	mMTC
每个用户的资源块占用量	10	5	2
SLA: 速率	10Mbps	50Mbps	1Mbps
SLA: 时延	5ms	100 ms	500 ms
任务数据量	0.01-0.012Mbit	0.1-0.12Mbit	0.012-0.014Mbit
任务到达分布	泊松分布	均匀分布	均匀分布
惩罚系数 M	5	3	1

(4) 目标函数

本题要求使用户服务质量达到最大, 故以最大化服务质量 QoS 为目标函数。

$$\max \quad QoS = Q_{URLLC} + Q_{eMBB} + Q_{mMTC} \quad (8)$$

因为仅有 16 个任务, 且各切片的惩罚系数均大于其服务质量的极大值, 应当避免受到惩罚。因此目标函数 (8) 中三个用户服务质量由方程组 (7) 得到:

$$\begin{cases} Q_{URLLC} = \sum_{i=1}^{x_u} \alpha^{L_i}; (\alpha = 0.95) \\ Q_{eMBB} = \sum_{j=1}^{x_e} \frac{r_j}{50} \\ Q_{mMTC} = \sum_{k=1}^{x_m} \frac{\sum_{i \in \mathcal{J}} c_i^j}{\sum_{i \in \mathcal{J}} c_i} \end{cases} \quad (9)$$

(5) 约束条件

$$s.t. \begin{cases} \text{资源块总数及每个用户的资源块占用量约束:} \\ 10x_u + 5x_e + 2x_m \leq 50 \\ \text{决策变量正整数约束:} \\ x_u, x_e, x_m \in \mathbb{Z}^+ \\ \text{各决策变量范围约束:} \\ 1 \leq x_u \leq 2 \\ 1 \leq x_e \leq 4 \\ 1 \leq x_m \leq 10 \\ \text{不受惩罚约束:} \\ \forall L \leq L_{SLA} \end{cases} \quad (10)$$

关于约束条件的说明:

1. 本题中仅有一个微基站，其资源块总数为 50 个。三种切片的每个用户的资源块占用量在附录中给出，URLLC 一用户 10 个，eMBB 一用户 5 个，mMTC 一用户 2 个。
2. 决策变量含义是分给各切片的资源块可覆盖的用户数，本文仅考虑可覆盖的用户数为正整数的情况。
3. 因为附件中所给的用户数固定，故各切片可覆盖的用户数不能大于其实际用户数，否则造成资源浪费。本题只进行一次决策，要在一个决策下完成传输任务，各切片可覆盖的用户数至少为 1。
4. 为简化本题模型，我们根据本题数据分析发现各切片都不受惩罚的条件易于实现，且惩罚系数较大，故认为最优解在各切片都不受惩罚对应结果域内。

5.1.2 模型求解

求解算法：分析缩域穷举搜索

由于变量维度低，附件 1 数据较少，可通过枚举法暴力求解可行方案。我们可以遍历所有满足约束条件的非负整数三元组 (x_u, x_e, x_m) ，计算出对应的总用户服务质量 QoS 。最后，选取 QoS 最大的组合作为最优解。

step1: 计算各用户任务的传输速率 r 和传输延迟 T 。由于计算 r 和 T 的各项参数固定， r 和 T 为定值。

step2: 根据所得 T ，分析满足不受惩罚约束下的各切片的决策变量取值范围，缩小穷举范围。其中 x_u 取值范围为 1、2； x_e 取值范围为 2、3、4； x_m 取值范围为 1~10。

step3: 根据方程组 (11)，只有 URLLC 的服务质量随总时延 L 增长而指数衰减，其余两种切片服务质量与总时延无关，因此固定 x_u 为 2。 x_e 取值范围为 2、3、4； x_m 取值范围为 1~10。

step4: 此时三种切片的总服务质量已经达到最大，但根据三种切片的特点（URLLC 极低的时延和高可靠性；eMBB 更快速、更稳定；mMTC 是大规模设备的连接性、低功耗和低成本），优先把资源块给 eMBB 使用，令 $x_e = 4$ ，剩余资源块给 mMTC 切片。故最优结果为 $x_u = 2, x_e = 4, x_m = 5$

5.1.3 结果展示

表 2 问题一的各用户任务的传输速率与传输延迟

用户	U1	U2	e1	e2	e3	e4	m1	m2
r/Mbps	7.1028	10.0783	5.4217	5.1686	3.9766	5.7878	1.9524	1.9607
T/ms	1.4508	1.0991	20.1749	21.8236	25.2867	34.2232	6.2327	6.3854
用户	m3	m4	m5	m6	m7	m8	m9	m10
r/Mbps	1.7218	2.1305	2.0233	2.3068	2.1489	1.9333	1.7815	1.9062
T/ms	7.4702	5.8029	6.0745	5.9555	6.0958	7.0891	7.1298	6.7168

通过分析，当 $x_u = 2, x_e = 4, x_m = 5$ 时，用户服务质量达到最优，为 12.2806。

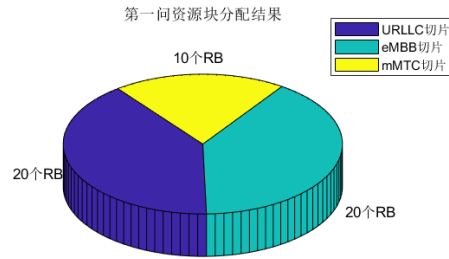


图 1 问题一最优资源块分配饼状图

5.2 问题二：动态累计任务各切片资源块分配优化模型

5.2.1 模型建立

(1) 决策时刻定义

假设每 100ms 进行一次决策，1000ms 内进行 10 次决策，决策时刻为 $t_{1,2,\dots,10} = 0, 100ms, \dots, 900ms$

(2) 决策变量

将本题的资源块最佳分配方案问题转化为求解在每个决策时刻 t 的三个决策变量，决策变量为分配给三类切片能覆盖的用户数 $(x_u(t), x_e(t), x_m(t))$ 。

(3) 用户 QoS 计算

将附录第三节信息整理得到三种切片的用户服务质量函数如下。对于任一切片用户，只要 $L > L_{SLA}$ ，就会受到惩罚，即服务质量为惩罚系数的相反数。对于 URLLC 用户其服务质量随 L 增大呈指数衰减，最大不超过 1。对于 eMBB 用户，其服务质量由传输速率与 SLA 规定传输速率的比值决定，最大不超过 1。对于 mMTC 用户，其服务质量由各用户是否完成传输任务决定，最大不超过 1。

$$\begin{cases} y^{URLLC} = \begin{cases} 0.95^L, & L \leq 5 \text{ ms} \\ -5, & L > 5 \text{ ms} \end{cases} \\ y^{eMBB} = \begin{cases} 1, & r \geq 50 \text{ Mbps 且 } L \leq 100 \text{ ms} \\ \frac{r}{50}, & r < 50 \text{ Mbps 且 } L \leq 100 \text{ ms} \\ -3, & L > 100 \text{ ms} \end{cases} \\ y^{mMTC} = \begin{cases} \frac{\sum c'_k}{\sum c_k}, & L \leq 500 \text{ ms} \\ -1, & L > 500 \text{ ms} \end{cases} \end{cases} \quad (11)$$

(4) 目标函数

本题虽然仍求使得整体的用户服务质量达到最大的三类切片的资源块最佳分配方案，但有 10 次决策，把最大化所有用户 10 次累积服务质量作为目标函数：

$$\max \sum_{t=1}^{10} \sum_{k=1}^{16} y_k^s(t) \quad (12)$$

其中，切片类型 $s \in \{URLLC, eMBB, mMTC\}$; 用户数 $k \in [1, 16]$ 且 $k \in \mathbb{Z}^+$; 决策时刻 $t \in [1, 10]$ 且 $t \in \mathbb{Z}^+$ 。

(5) 约束条件

对于每个决策时刻 t

$$s.t. \begin{cases} \text{资源分配约束:} \\ 10x_u + 5x_e + 2x_m \leq 50 \\ \text{整数约束:} \\ x_u, x_e, x_m \in \mathbb{Z}^+ \\ \text{接入约束 (mMTC):} \\ c'_k \in \{0, 1\} \quad (100\text{ms 内成功接入} = 1) \end{cases} \quad (13)$$

约束条件说明：

因为本题 mMTC 切片任务量增大后，很可能出现 100ms 内也未接入的情况，接入约束 (mMTC) 主要用于计算 mMTC 切片用户服务质量。

5.2.2 模型求解

在求解问题二模型时，我们采用**贪心算法**思想，寻找每个决策时刻 t 的局部最优解，从而得到全局次优解。

1. 在决策时刻 t 开始时，更新任务队列。

新任务队列包括原任务、加入新到达任务、去除已完成任务；采集当前信道状态。

2. 优化资源分配。

采用问题一类似的算法遍历所有可能的组合，生成可行解空间；

分层剪枝：第一层，优先保证 URLLC 满足约束。第二层，其次保证 eMBB 满足约束。第三层，调整分配平衡 mMTC。

3. 以 1ms 为步长模拟 100ms 传输。

对每一种可行方案，模拟执行其 100ms 的服务过程，得出：能够完成任务的队列、这些任务的总 QoS、未完成（被惩罚）任务排队情况。

4. 记录此时此刻当总 QoS 最大时的决策方案。记录内容包括：决策时刻，分配方案，QoS 值。

5. 判断是否完成 10 次决策。

若完成，则输出所有决策方案作为最佳方案；否则，进入下一个决策时刻。

问题二算法的 2,3,4 步详细流程如下图所示：

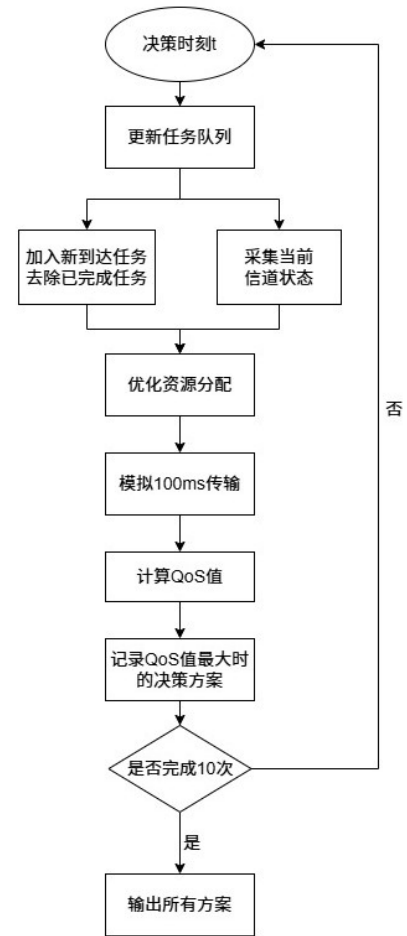


图 2 问题二算法流程图

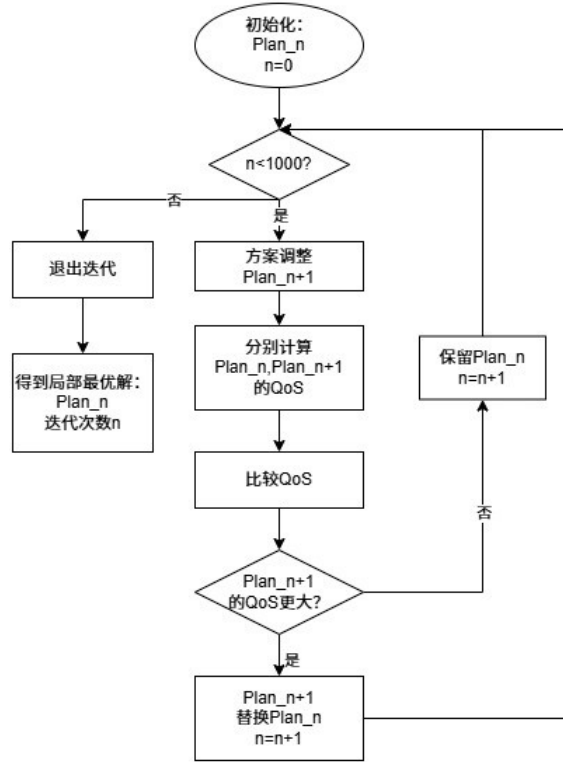


图3 贪心算法详细流程图

5.2.3 结果展示

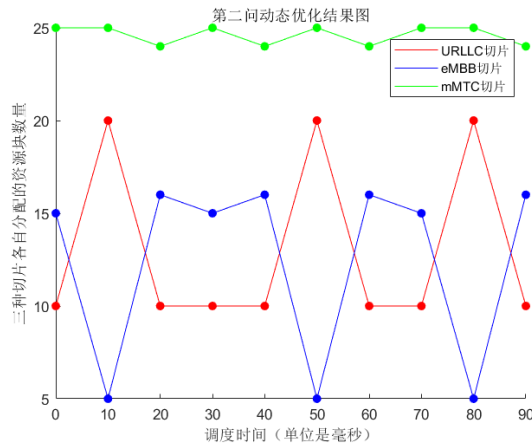


图4 问题二结果图

5.3 问题三：多个微基站资源块分配优化和功率控制优化模型

5.3.1 模型建立

(1) 决策变量

本题已知有三个基站，不考虑一个用户的一个任务有以下三种决策变量：

1. $a_{k,n,s,t} \in \{0,1\}$: 在决策时刻 t , 用户 k 是否被分配至基站 n 的切片 s
2. $x_{n,s,t}$: 在决策时刻 t , 基站 n 分配给切片 s 的资源块能覆盖的用户数
3. $p_{n,s,t}$: 在决策时刻 t , 基站 n 切片 s 的发射功率, 单位 dBm

其中, $n \in \{BS1, BS2, BS3\}, s \in \{URLLC, eMBB, mMTC\}$

(2) 关键变量计算

接收信号功率 (单位: mW)

$$p_{rx} = 10^{\frac{p_{n,k} - \phi_{n,k}}{10}} \cdot |h_{n,k}|^2$$

白噪声 (单位: dBm, 转换为: mW)

$$N_0(\text{dBm}) = -174 + 10 \log_{10}(ib) + NF$$

$$N_0(\text{mW}) = 10^{\frac{N_0(\text{dBm})}{10}}$$

信干噪比 (SINR): 存在干扰项, 故对于一个连接到基站的用户 k , 其 SINR 为

$$\gamma_{k,n,t} = \frac{p_{n,s,t} \phi_{n,k,t} |h_{n,k,t}|^2}{\sum_{\substack{u \in \mathcal{N} \\ u \neq n}} p_{u,s',t} \phi_{u,k,t} |h_{u,k,t}|^2 + N_0(mW)} = \frac{10^{\frac{p_{n,s,t} - \phi_{n,k,t}}{10}} \cdot |h_{n,k,t}|^2}{\sum_{\substack{u \in \mathcal{N} \\ u \neq n}} 10^{\frac{p_{u,s',t} - \phi_{u,k,t}}{10}} \cdot |h_{u,k,t}|^2 + N_0(mW)}$$

传输速率 (单位: bps)

$$r_{k,t} = i_k b \times \log_2(1 + \gamma_{k,n,t})$$

用户 k 在时刻 t 的总延迟 (单位: s)

$$L_{k,t} = Q_{k,t} + T_{k,t}$$

(14)

(3) 用户 QoS 计算与问题二中的方程组 (11) 相同

(4) 目标函数

目标函数依然是最大化所有用户累积服务质量:

$$\max \sum_{t=1}^{10} \sum_{k=1}^{48} y_k^s(t) \quad (15)$$

其中, $s \in \{URLLC, eMBB, mMTC\}$

(5) 约束条件

$$\begin{aligned}
 & \text{资源块分配约束:} \\
 & 10x_{b,u,t} + 5x_{b,e,t} + 2x_{b,m,t} \leq 50, \quad \forall b, \quad t = 1, \dots, 10 \\
 & \text{唯一分配:} \\
 & \sum_b \sum_s x_{k,b,s,t} \leq 1, \quad \forall k, \quad t = 1, \dots, 10 \\
 & \text{整数约束:} \\
 & x_u, x_e, x_m \in Z^+ \\
 & \text{功率范围约束:} \\
 & 10 \leq p_{b,s,t} \leq 30, \quad \forall b, \quad \forall s, \quad t = 1, \dots, 10 \\
 & \text{信干噪比 (SINR) 约束:} \\
 & \gamma_{k,n,t} \geq \gamma_{min} \\
 & \text{接入约束 (mMTC):} \\
 & c'_k \in \{0, 1\} \quad (\text{成功接入} = 1)
 \end{aligned}
 \tag{16}$$

5.3.2 模型求解

由于此模型复杂，为混合整数非线性规划，且需要实时决策（每 100ms）动态规划，我们采用分层迭代优化算法，将模型分解为资源分配和功率控制两个子模型，模拟服务过程计算 QoS，迭代优化。

(1) 数据预处理

1. 固定每个基站的功率，分别为 $p_1, p_2, p_3, p_i \in [10, 30]dBm, i = 1, 2, 3$
2. 分别计算用户 k 在无干扰和有干扰情况下不同基站的时延 $T_{k,n}$
3. 用户选择 $\min T_{k,n}$ 所在基站作为接入基站
4. 分别累计得出 URLLC, eMBB, mMTC 对有干扰和无干扰时对每个基站的接入次数和
5. 改变功率，再次进行上述计算
6. 比较数据得到结论：

有无干扰对用户选择没有影响，即对每个基站的用户均衡性没有影响；
功率改变导致干扰程度改变，时延也因此改变，故影响用户均衡性。

(2) 算法求解

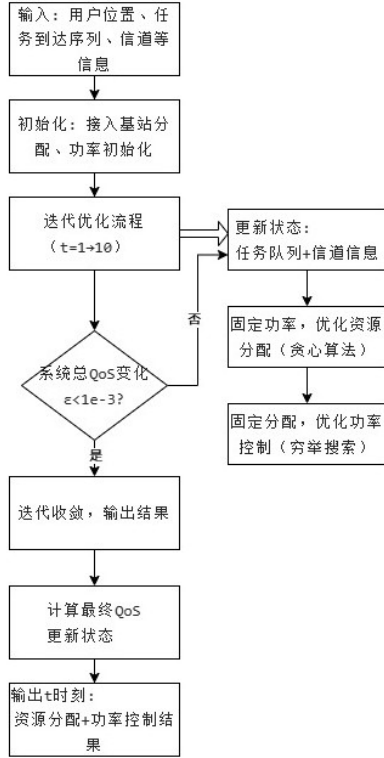


图 5 问题三算法流程图

1. **输入**：决策时刻 t 的任务队列，信道信息等

2. **初始化**：

- 每个用户 k 接入时延最小的基站 n ；
- 将基站 n 切片 s 的发射功率设为最大值 30dBm

3. **迭代优化流程**：(对每个决策时刻 $t = 1, \dots, 10$)

- 更新状态。更新任务队列：加入新到达任务、去除已完成任务；采集并计算当前信道信息。
- 固定功率，优化资源分配。采用问题二的贪心算法思想分层分配 RB。
- 固定分配，优化功率控制。
- 迭代收敛。重复以上步骤，直至系统总 QoS 不再发生显著变化，即变化小于阈值 $\epsilon = 10^{-3}$
- 结果输出。计算最终 QoS，更新状态。

4. **资源分配子问题**

目标函数： $\max \sum_k y_k^s$ ，固定功率 $p_{n,s,t}$

算法：

- 将任务按照切片优先级排序：URLLC > eMBB > mMTC。同一切片类型按用户序号升序排列。
- 遍历形成可行分配组合，对每一种可行方案，模拟 100ms 服务过程。
- 记录最大 QoS 时的决策方案。

5. **功率控制子问题**

目标函数： $\max \sum_k y_k^s$ ，固定资源分配 $x_{n,s,t}, a_{k,n,s,t}$

算法：

- 按 0.1dBm 步长将功率范围离散化；生成 9 维功率空间组合矩阵（3 基站 × 3 切片）
- 对于每个功率组合，计算 QoS 值
- 比较所有组合的 QoS，选择最大值对应的功率分配方案。

6. **输出**：每次决策时刻 t 的资源块分配 $\{x_{n,s,t}\}$ 、功率控制 $\{p_{n,s,t}\}$

5.3.3 结果展示

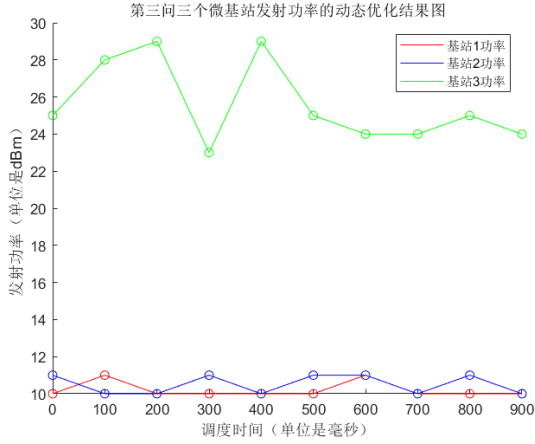


图6 问题三发射功率优化结果

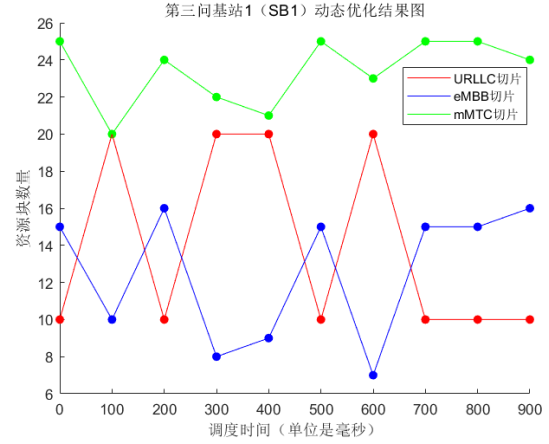


图7 问题三基站1资源块优化结果

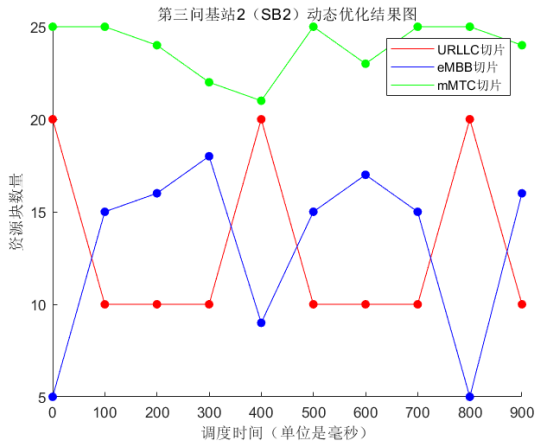


图8 问题三基站2资源块优化结果

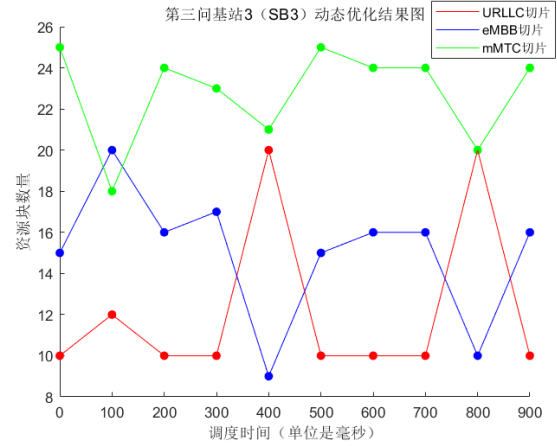


图9 问题三基站3资源块优化结果

5.4 问题四：用户接入决策，切片划分决策以及发射功率分配优化模型

5.4.1 模型建立

(1) 决策变量

- $a_{k,n,s,t} \in \{0,1\}$: 在决策时刻 t ，用户 k 是否被分配至基站 n 的切片 s ，若被分配至 n ，则为 1；反之，则为 0
- $x_{n,s,t}$: 在决策时刻 t ，基站 n 分配给切片 s 的资源块能覆盖的用户数
- $p_{n,s,t}$: 在决策时刻 t ，基站 n 切片 s 的发射功率，单位 dBm。

其中， $n \in \{SBS1, SBS2, SBS3, MBS1\}$, $s \in \{URLLC, eMBB, mMTC\}$

(3) 用户 QoS 计算与问题二中的方程组 (11) 相同

(4) 关键变量计算

接收信号功率 (单位: mW)

$$p_{rx} = 10^{\frac{p_{n,k} - \phi_{n,k}}{10}} \cdot |h_{n,k}|^2$$

白噪声计算式 (单位: dBm, 转换为: mW)

$$N_0(\text{dBm}) = -174 + 10 \log_{10}(ib) + NF$$

$$N_0(\text{mW}) = 10^{\frac{N_0(\text{dBm})}{10}}$$

其中, i_k 是用户 k 所占的 RB 数

信干噪比 ($\gamma_{k,MBS,t}$): 存在干扰项, 故对于一个连接到基地 n 的用户 k , 其信干噪比为

$$\left\{ \begin{array}{l} \text{接入宏基站: } \gamma_{k,MBS,t} = \frac{10^{\frac{p_{MBS,k} - \phi_{MBS,k}}{10}} \cdot |h_{MBS,k}|^2}{N_0(\text{mW})} (\text{无干扰}) \\ \text{接入微基站: } \gamma_{k,n,t} = \frac{10^{\frac{p_{n,s,t} - \phi_{n,k,t}}{10}} \cdot |h_{n,k,t}|^2}{\sum_{\substack{u \in \mathcal{N} \\ u \neq n}} 10^{\frac{p_{u,s',t} - \phi_{u,k,t}}{10}} \cdot |h_{u,k,t}|^2 + N_0(\text{mW})} (\text{存在其他微基站干扰}) \end{array} \right.$$

其中, s' 是干扰基站 u 的切片类型

传输速率计算式 (单位: bps)

$$r_{k,t} = i_k b \times \log_2(1 + \gamma_{k,n,t})$$

用户 k 在时刻 t 的总延迟 (单位: s)

$$L_{k,t} = Q_{k,t} + T_{k,t}$$

(17)

(5) 目标函数

目标函数依然是最大化所有用户累积服务质量:

$$\max \sum_{t=1}^{10} \sum_{k=1}^{70} y_k^s(t) \quad (18)$$

其中, $s \in \{URLLC, eMBB, mMTC\}$

(6) 约束条件

$$\begin{cases}
\text{用户接入约束:} \\
\sum_n a_{k,n} = 1, \quad \forall k \\
\text{资源块分配约束:} \\
\begin{cases}
\text{宏基站: } 10x_{MBS,u,t} + 5x_{MBS,e,t} + 2x_{MBS,m,t} \leq 100, \quad \forall b, \quad t = 1, \dots, 10 \\
\text{微基站: } 10x_{SBS_i,u,t} + 5x_{SBS_i,e,t} + 2x_{SBS_i,m,t} \leq 50, \quad \forall b, \quad \forall i \in \{1, 2, 3\}, \quad t = 1, \dots, 10
\end{cases} \\
\text{功率范围约束:} \\
\begin{cases}
\text{宏基站: } 10 \leq p_{MBS} \leq 40, \quad t = 1, \dots, 10 \\
\text{微基站: } 10 \leq p_{SBS_i} \leq 30, \quad \forall i \in \{1, 2, 3\}, t = 1, \dots, 10
\end{cases}
\end{cases} \tag{19}$$

5.4.2 模型求解

由于此模型为混合整数非线性规划,且变量维度高,直接求解难度大,我们仍采用问题三的算法求解——分层迭代优化算法,将此模型分解为用户接入决策、资源块划分决策和发射功率控制决策等三层,模拟服务过程计算 QoS,迭代优化。

(1) 数据预处理

1. 比较用户位置变化发现其变化微小,因此根据第 0ms 的位置信息进行下一步计算
2. 根据欧几里得距离公式 $d = \sqrt{(x_{\text{user}} - x_{\text{base}})^2 + (y_{\text{user}} - y_{\text{base}})^2}$, 计算各个用户与每个微基站的距离,比较得到最近的微基站,整理得到最近微基站矩阵。
3. 固定每个基站的功率,分别为 p_1, p_2, p_3, p_4 $p_i \in [10, 30] \text{ dBm}, i = 1, 2, 3; p_4 \in [10, 40]$
4. 分别计算用户 k 在无干扰和有干扰情况下不同基站的时延 $L_{k,n}$

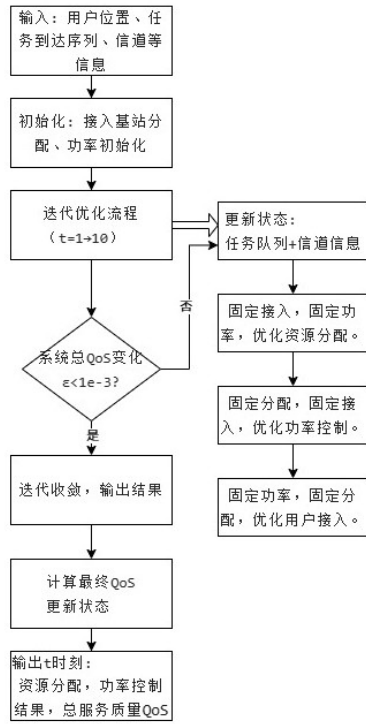


图 10 问题四算法流程图

(2) 算法求解

1. **输入:** 用户位置、任务到达时间序列、信道信息、基站位置、SLA 参数等。

2. **初始化:**

- 用户 k 选择宏基站和最近的微基站中时延较小的基站接入
- 将基站 n 切片 s 的发射功率均设为最大值

3. **迭代优化流程:** (对每个决策时刻 $t = 1, \dots, 10$)

- 更新状态。更新任务队列: 加入新到达任务、去除已完成任务; 采集并计算当前信道信息。
- 固定接入, 固定功率, 优化资源分配。采用问题二的贪心算法思想分层分配 RB。
- 固定分配, 固定接入, 优化功率控制。
- 固定功率, 固定分配, 优化用户接入。
- 迭代收敛。重复以上步骤, 直至系统总 QoS 不再发生显著变化, 即变化小于阈值 $\epsilon = 10^{-3}$
- 结果输出。计算最终 QoS, 更新状态。

4. 资源分配子问题:

目标函数: $\max \sum_k y_k^s$, 固定接入 $a_{k,n,s,t}$, 固定功率 $p_{n,s,t}$

算法:

- 将任务按照切片优先级排序: URLLC > eMBB > mMTC。同一切片类型按用户序号升序排列。
- 遍历形成可行分配组合, 对每一种可行方案, 模拟 100ms 服务过程。
- 记录最大 QoS 时的决策方案。

5. 功率控制子问题:

目标函数: $\max \sum_k y_k^s$, 固定资源分配 $x_{n,s,t}$, 固定接入 $a_{k,n,s,t}$

算法:

- 按 0.1dBm 步长将功率范围离散化; 生成 12 维功率空间组合矩阵 (4 基站 $\times$ 3 切片)
- 对于每个功率组合, 计算 QoS 值
- 比较所有组合的 QoS, 选择最大值对应的功率分配方案。

6. 用户接入决策子问题:

目标函数: $\max \sum_k y_k^s$, 固定功率 $p_{n,s,t}$, 固定资源分配 $x_{n,s,t}$

算法：

- 对于每个用户，分别计算他接入 MBS 和最近的 SBS 的 QoS 值
- 比较 QoS 大小，选择最大值对应的基站重新接入。

7. 输出：

- 每个用户的接入基站
- 各基站切片分配 RB 数量
- 各基站发射功率
- 10 次决策的总服务质量

5.4.3 结果展示

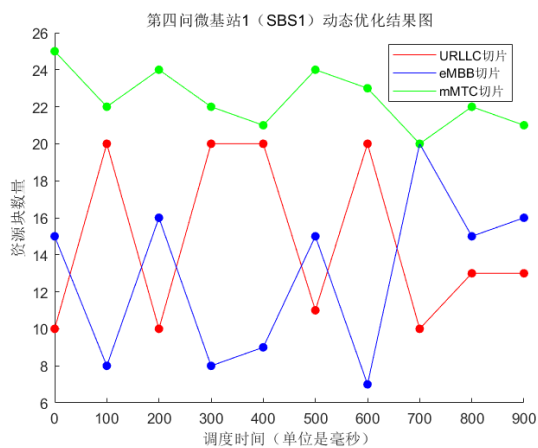


图 11 问题四微基站 1 资源块优化结果

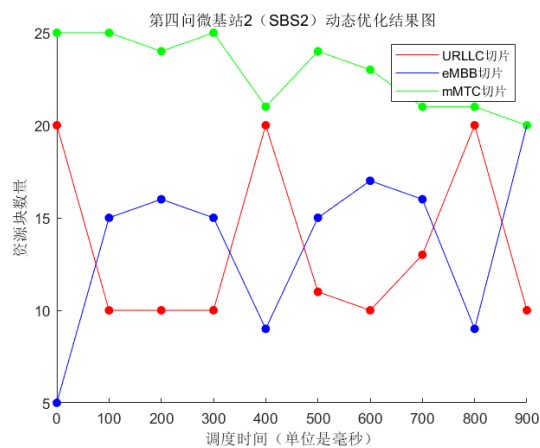


图 12 问题四微基站 2 资源块优化结果

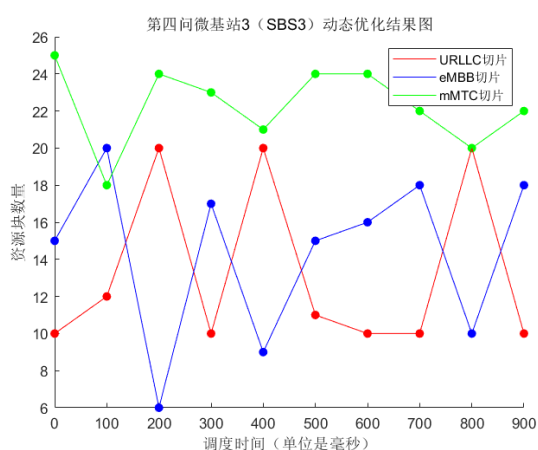


图 13 问题四微基站 3 资源块优化结果

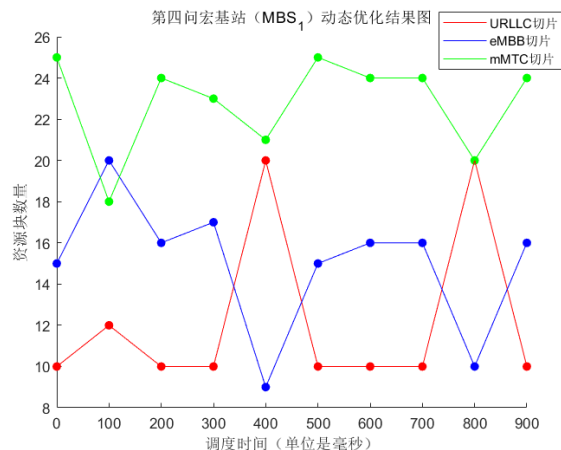


图 14 问题四宏基站资源块优化结果

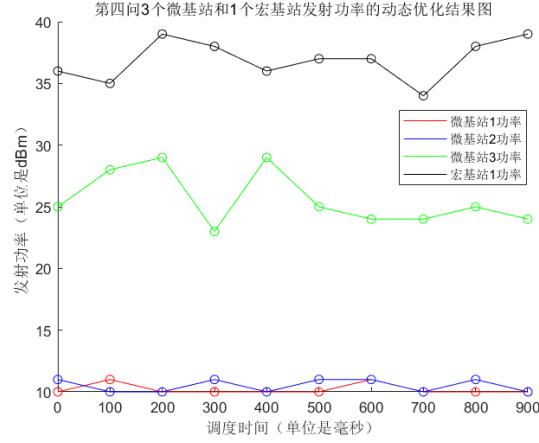


图 15 问题四各基站功率优化结果

5.5 问题五：能耗和质量双目标优化模型

5.5.1 模型建立

(1) 决策变量

- $a_{k,n,s,t} \in \{0, 1\}$: 在决策时刻 t , 用户 k 是否被分配至基站 n 的切片 s , 若被分配至 n , 则为 1; 反之, 则为 0
- $x_{n,s,t}$: 在决策时刻 t , 基站 n 分配给切片 s 的资源块能覆盖的用户数
- $p_{n,s,t}$: 在决策时刻 t , 基站 n 切片 s 的发射功率, 单位 dBm。

其中, $n \in \{SBS1, SBS2, SBS3, MBS1\}, s \in \{URLLC, eMBB, mMTC\}$

(2) 能耗模型

$$\left\{ \begin{array}{l} \text{单个基站 } n \text{ 的能耗为:} \\ P_n = P_{\text{static}} + P_{\text{RB}} + P_{\text{tx}} \\ \text{其中} \left\{ \begin{array}{l} n \in \{SBS1, SBS2, SBS3, MBS1\} \\ P_{\text{static}} (\text{单位 W}): \text{固定能耗, 为 } 28\text{W} \\ P_{\text{RB}} (\text{单位 W}): \text{RB 激活能耗, } P_{\text{RB}} = \delta \times N_{\text{active}} \\ \text{其中 } \delta = 0.75 (\text{W/RB}) \text{ 表示激活能耗系数, } N_{\text{active}} \text{ 为资源块启动数量。} \\ P_{\text{tx}} (\text{单位 dBm, 转化为 W}): \text{发射能耗, } P_{\text{tx}} = \frac{1}{\eta} p_{\text{transmit}} \\ \text{其中 } \eta = 0.35 \text{ 为损耗系数, } p_{\text{transmit}} \text{ 为发射功率} \\ \text{单位转化公式: } p_{(\text{W})} = 10^{\frac{p_{(\text{dBm})} - 30}{10}} \end{array} \right. \end{array} \right. \quad (20)$$

(3) 用户 QoS 计算与问题二中的方程组 (11) 相同

(4) 目标函数 最小化总能耗:

$$\min E \quad (21)$$

(5) 约束条件

$$\left\{ \begin{array}{l} \text{服务质量约束: 总服务质量需不小于第四问的最大值 } Q_{max}, \text{ 即} \\ \sum_{t=1}^{10} \sum_{k=1}^{70} y_k^s(t) \geq Q_{max} \\ \text{用户接入约束:} \\ \sum_n a_{k,n} = 1, \quad \forall k \\ \text{资源块分配约束:} \\ \left\{ \begin{array}{l} \text{宏基站: } 10x_{MBS,u,t} + 5x_{MBS,e,t} + 2x_{MBS,m,t} \leq 100, \quad \forall b, \quad t = 1, \dots, 10 \\ \text{微基站: } 10x_{SBSi,u,t} + 5x_{SBSi,e,t} + 2x_{SBSi,m,t} \leq 50, \quad \forall b, \quad \forall i \in \{1, 2, 3\}, \quad t = 1, \dots, 10 \end{array} \right. \\ \text{功率范围约束:} \\ \left\{ \begin{array}{l} \text{宏基站: } 10 \leq p_{MBS} \leq 40, \quad t = 1, \dots, 10 \\ \text{微基站: } 10 \leq p_{SBS_i} \leq 30, \quad \forall i \in \{1, 2, 3\}, t = 1, \dots, 10 \end{array} \right. \end{array} \right. \quad (22)$$

5.5.2 模型求解

算法求解

1. 输入: 问题四的结果包括每个用户的接入基站、各基站切片分配 RB 数量、各基站发射功率和 10 次决策的最大总服务质量 Q_{max} 。
2. 优化功率控制:
 - 以 0.1dBm 步长逐步降低功率
 - 模拟服务过程, 检查是否满足约束条件和用户 QoS 模型
 - 计算总能耗, 比较是否降低, 若降低, 则更新功率; 否则保持原来功率。
3. 优化资源块分配:
 - 尝试减少资源块数量
 - 模拟服务过程, 检查是否满足约束条件和用户 QoS 模型
 - 若满足, 则减少资源块; 否则保持原来数量不变
4. 迭代终止: 重复以上步骤, 直至系统总能耗 E 不再发生显著变化, 即变化小于阈值 $\epsilon = 10^{-3}$
5. 输出: 优化后的分配、总服务质量以及最小能耗 E_{min}

5.5.3 结果展示

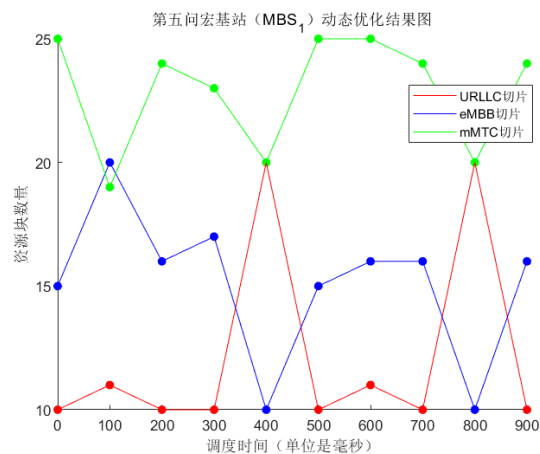


图 16 问题五宏基站资源块优化结果

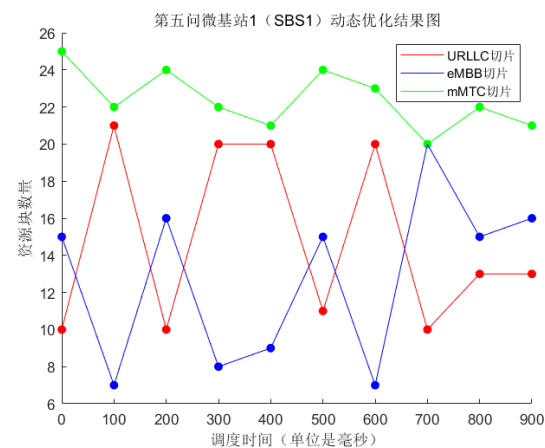


图 17 问题五各基站发射功率优化结果

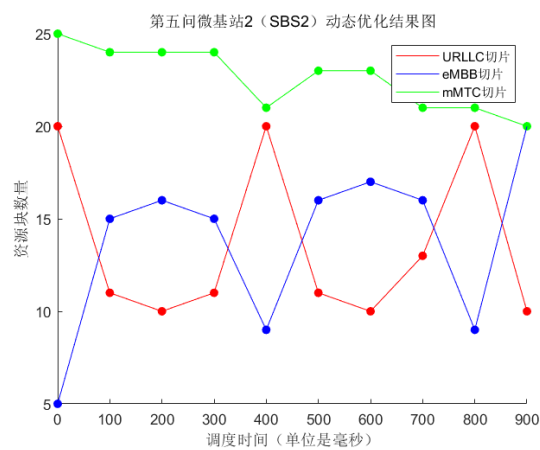


图 18 问题五基站 2 资源块优化结果

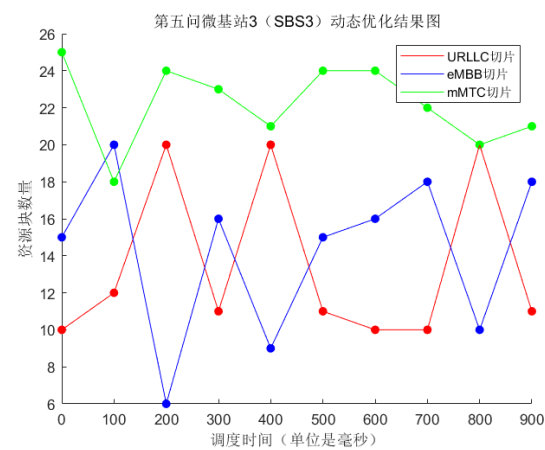


图 19 问题五微基站 3 资源块优化结果

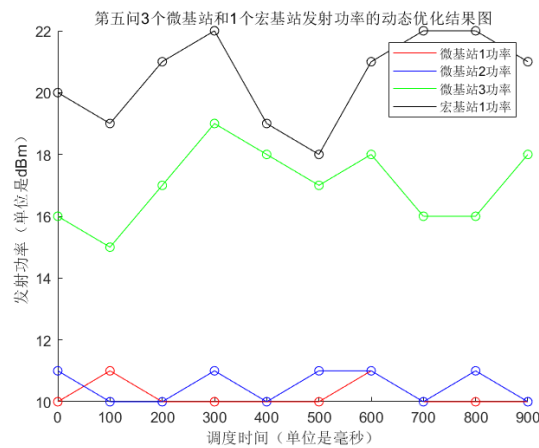


图 20 问题五微基站 3 资源块优化结果

六、模型评价

6.1 模型优点

1. 本文在正确、清楚的分析 5G 网络信息传输流程的基础上，将模型进行合理简化，为优化服务质量准备了条件。
2. 模型求解采用合适的算法，例如问题一的分析缩域的穷举搜索，以及后续问题的动态优化算法，节约算力和时间，同时保证准确性。

6.2 模型缺点

1. 本文所建立的模型未考虑现实生活中用户需求为概率分布的情况，仅能应用于已知任务具体分布情况的情形。

七、模型改进

可以将附件中的数据转化为概率模型，例如 URLLC 用户数据为泊松分布，参照排队模型和排队论方法建立类似的混合整数非线性优化模型。

参考文献

- [1] Tidiane S ,Leo M ,Sassi M , et al.Multi-Connectivity for 5G Networks and Beyond: A Survey[J].Sensors,2022,22(19):7591-7591.

附录 A 支撑材料文件列表

文件名	文件内容
problem_1.m	问题一求解
problem_1_plot.m	问题一用户位置坐标图
problem_1_result_show.m	问题一结果分析图
problem_2.m	问题二求解
problem_2_e_score_plot.m	问题二 eMBB 的 RB 分配与 QoS 的关系
problem_2_m_score_plot.m	问题二求解 mMTC 的 RB 分配与 QoS 的关系
problem_2_U_score_plot.m	问题二 URLLC 的 RB 分配与 QoS 的关系
problem_2_result_show.m	问题二结果分析图
problem_3.m	问题三求解
problem_3_plot.m	问题三用户位置坐标图
problem_3_result_show.m	问题三结果分析图
problem_4.m	问题四求解
problem_4_plot.m	问题四用户位置坐标图
problem_4_result_show.m	问题四结果分析图
problem_5.m	问题五求解
problem_5_result_show.m	问题五结果分析图
t2_e_score.m	问题二 eMBB 的总 QoS
t2_e_score_100ms.m	问题二 eMBB 每 100ms 的 QoS
t2_m_score.m	问题二 mMTC 的总 QoS
t2_m_score_100ms.m	问题二 mMTC 每 100ms 的 QoS
t2_U_score.m	问题二 URLLC 的总 QoS
t2_U_score_100ms.m	问题二 URLLC 每 100ms 的 QoS
t3_p_count.m	问题三不同功率下各基站的结果分析
t4_MBS_SBS.xlsx	问题四对四个基站的选择
t4_p_count.m	问题四不同功率下各基站的结果分析

附录 B problem_1

```
clc,clear,close all

format short

filePath = fullfile('附件', '附件1', 'channel_data.xlsx');

phi = readmatrix(filePath, 'Sheet', '大规模衰减', "Range", "B2:Q2"); % U2 e4 m10
U_phi = phi(1:2);
e_phi = phi(3:6);
m_phi = phi(7:16);
h = readmatrix(filePath, 'Sheet', '小规模瑞丽衰减', "Range", "B2:Q2");
U_h = h(1:2);
e_h = h(3:6);
m_h = h(7:16);
taskflow = readmatrix(filePath, 'Sheet', '任务流', "Range", "B2:Q2");
U_taskflow = taskflow(1:2);
e_taskflow = taskflow(3:6);
m_taskflow = taskflow(7:16);
axis_xy = readmatrix(filePath, 'Sheet', '用户位置', "Range", "B2:AG2");
U_axis_xy = axis_xy(1:4);
e_axis_xy = axis_xy(5:12);
m_axis_xy = axis_xy(13:32);
p = 30; % dbm
b = 360000; % Hz
U_i = 10; % RB占用量
e_i = 5;
m_i = 2;
NF = 7;
alpha = 0.95;
r_LSA = 50; % Mbps

U_N0 = -174+10.*log10(U_i.*b)+NF;
U_N0 = 10.^(U_N0/10);
U_p_rx = (10.^((p-U_phi)./10)).*(abs(U_h)).^2;
U_Gamma = ( U_p_rx ) ./ (U_N0);
U_r = U_i.*b.*log2(1+U_Gamma) ./10e6 % Mbps bps = bit/s
U_t = 1000.*U_taskflow./U_r % 毫秒 ms

e_N0 = -174+10.*log10(e_i.*b)+NF;
e_N0 = 10.^(e_N0/10);
e_p_rx = (10.^((p-e_phi)./10)).*(abs(e_h)).^2;
e_Gamma = ( e_p_rx ) ./ (e_N0);
e_r = e_i.*b.*log2(1+e_Gamma) ./10e6 % Mbps bps = bit/s
e_t = 1000.*e_taskflow./e_r % 毫秒 ms
```

```

m_NO = -174+10.*log10(m_i.*b)+NF;
m_NO = 10.^(m_NO/10);
m_p_rx = (10.^(p-m_phi)./10)).*(abs(m_h)).^2;
m_Gamma = ( m_p_rx ) ./ (m_NO);
m_r = m_i.*b.*log2(1+m_Gamma) ./10e6 % Mbps bps = bit/s
m_t = 1000.*m_taskflow./m_r          % 毫秒 ms

U_score = alpha.^U_t
e_score = e_r./r_LSA
m_score = ones(1,10)

U_total_score = sum(U_score)
e_total_score = sum(e_score)
m_total_score = sum(m_score)

total_score = U_total_score + e_total_score + m_total_score %20 20 10

```

附录 C problem_1_plot

```

clc,clear,close all

format short

filePath = fullfile('附件', '附件1', 'channel_data.xlsx');

axis_xy = readmatrix(filePath,'Sheet','用户位置',"Range","B2:AG2");

hold on

for i = 1:16
    scatter(axis_xy(2*i-1),axis_xy(2*i),15,"r","filled")
end
% scatter(0, 0, 15,"b","filled")
labels = {'U1', 'U2', 'e1', 'e2', 'e3', 'e4', 'm1', 'm2', 'm3',
          'm4', 'm5', 'm6', 'm7', 'm8', 'm9', 'm10'};

% 在每个点上添加标注
for i = 1:16
    text(axis_xy(2*i-1), axis_xy(2*i), labels{i}, 'VerticalAlignment', 'bottom',
         'HorizontalAlignment', 'right');
end
% text(0, 0, 'BS', 'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'right');

xlabel("X轴")

```

```
ylabel("Y轴")
title("第一问和第二问用户位置")
```

附录 D problem_1_result_show

```
clc,clear,close all

data = [20, 20, 10];
labels = {'20个RB', '20个RB', '10个RB'}; % 标注类别及占比

pie3(data, labels);
title('第一问资源块分配结果');
legend("URLLC切片","eMBB切片","mMTC切片")
```

附录 E problem_2

```
clc,clear,close all

format short g

filePath = fullfile('附件', '附件2', 'channel_data.xlsx');

phi = readmatrix(filePath, 'Sheet', '大规模衰减', "Range", "B2:Q1001"); % U2 e4 m10
U_phi = phi(:,1:2);
e_phi = phi(:,3:6);
m_phi = phi(:,7:16);
h = readmatrix(filePath, 'Sheet', '小规模瑞丽衰减', "Range", "B2:Q1001");
U_h = h(:,1:2);
e_h = h(:,3:6);
m_h = h(:,7:16);
taskflow = readmatrix(filePath, 'Sheet', '用户任务流', "Range", "B2:Q1001");
U_taskflow = taskflow(:,1:2);
e_taskflow = taskflow(:,3:6);
m_taskflow = taskflow(:,7:16);
axis_xy = readmatrix(filePath, 'Sheet', '用户位置', "Range", "B2:AG1001");
U_axis_xy = axis_xy(:,1:4);
e_axis_xy = axis_xy(:,5:12);
m_axis_xy = axis_xy(:,13:32);
p = 30; % dbm
b = 360000; % Hz
U_i = 10; % RB占用量
e_i = 5;
```

```

m_i = 2;
NF = 7;
alpha = 0.95;
e_r_LSA = 50; % Mbps

U_NO = -174+10.*log10(U_i.*b)+NF;
U_NO = 10.^(U_NO/10);
U_p_rx = (10.^((p-U_phi)./10)).*(abs(U_h)).^2;
U_Gamma = ( U_p_rx ) ./ (U_NO);
U_r = U_i.*b.*log2(1+U_Gamma) ./10e6 ; % Mbps bps = bit/s

U_t = 1000.*U_taskflow./U_r ; % 毫秒 ms

e_NO = -174+10.*log10(e_i.*b)+NF;
e_NO = 10.^(e_NO/10);
e_p_rx = (10.^((p-e_phi)./10)).*(abs(e_h)).^2;
e_Gamma = ( e_p_rx ) ./ (e_NO);

e_r = e_i.*b.*log2(1+e_Gamma) ./10e6 ; % Mbps bps = bit/s
e_t = 1000.*e_taskflow./e_r ; % 毫秒 ms

m_NO = -174+10.*log10(m_i.*b)+NF;
m_NO = 10.^(m_NO/10);
m_p_rx = (10.^((p-m_phi)./10)).*(abs(m_h)).^2;
m_Gamma = ( m_p_rx ) ./ (m_NO);
m_r = m_i.*b.*log2(1+m_Gamma) ./10e6 ; % Mbps bps = bit/s

m_t = 1000.*m_taskflow./m_r ; % 毫秒 ms

% [total_score,L_list] = t2_m_score(20,m_t);
% [total_score,L_list] = t2_e_score(10,e_t,e_r,e_r_LSA);
% [total_score,L_list] = t2_U_score(1,U_t);

uu = zeros(10,1);
for i = 1:10
    U_t100ms = U_t(100*(i-1)+1:100*i,:);
    [total_score,L_list] = t2_U_score_100ms(i,2,U_t100ms);
    L_list;
    for j = 1:100
        if any(L_list(j,:) > 101-j)
            L_list(j,:)
            i
        end
    end
end

```

```

        uu(i) = t2_U_score_100ms(i,2,U_t100ms)-t2_U_score_100ms(i,1,U_t100ms);
end
uu

% ee = zeros(10,1);
% for i = 1:10
%     e_t100ms = e_t(100*(i-1)+1:100*i,:);
%     e_r100ms = e_r(100*(i-1)+1:100*i,:);
%     ee(i) =
        t2_e_score_100ms(i,4,e_t100ms,e_r100ms,e_r_LSA)-t2_e_score_100ms(i,2,e_t100ms,e_r100ms,e_r_LSA);
% end
% ee

% m_t100ms = m_t(101:200,:);
% [total_score,L_list] = t2_m_score_100ms(1,m_t100ms)

% mm = zeros(10,1);
% for i = 1:10
%     m_t100ms = m_t(100*(i-1)+1:100*i,:);
%     mm(i) = t2_m_score_100ms(6,m_t100ms)-t2_m_score_100ms(1,m_t100ms);
% end
% mm

```

附录 F problem_2_e_score_plot

```

clc,clear,close all

format short g

filePath = fullfile('附件', '附件2', 'channel_data.xlsx');

phi = readmatrix(filePath, 'Sheet', '大规模衰减', "Range", "B2:Q1001"); % U2 e4 m10
U_phi = phi(:,1:2);
e_phi = phi(:,3:6);
m_phi = phi(:,7:16);
h = readmatrix(filePath, 'Sheet', '小规模瑞丽衰减', "Range", "B2:Q1001");
U_h = h(:,1:2);
e_h = h(:,3:6);
m_h = h(:,7:16);
taskflow = readmatrix(filePath, 'Sheet', '用户任务流', "Range", "B2:Q1001");
U_taskflow = taskflow(:,1:2);
e_taskflow = taskflow(:,3:6);
m_taskflow = taskflow(:,7:16);
axis_xy = readmatrix(filePath, 'Sheet', '用户位置', "Range", "B2:AG1001");
U_axis_xy = axis_xy(:,1:4);

```

```

e_axis_xy = axis_xy(:,5:12);
m_axis_xy = axis_xy(:,13:32);
p = 30; % dbm
b = 360000; % Hz
U_i = 10; % RB占用量
e_i = 5;
m_i = 2;
NF = 7;
alpha = 0.95;
e_r_LSA = 50; % Mbps

U_NO = -174+10.*log10(U_i.*b)+NF;
U_NO = 10.^(U_NO/10);
U_p_rx = (10.^((p-U_phi)./10)).*(abs(U_h)).^2;
U_Gamma = ( U_p_rx ) ./ (U_NO);
U_r = U_i.*b.*log2(1+U_Gamma) ./10e6 ; % Mbps bps = bit/s

U_t = 1000.*U_taskflow./U_r ; % 毫秒 ms

e_NO = -174+10.*log10(e_i.*b)+NF;
e_NO = 10.^(e_NO/10);
e_p_rx = (10.^((p-e_phi)./10)).*(abs(e_h)).^2;
e_Gamma = ( e_p_rx ) ./ (e_NO);

e_r = e_i.*b.*log2(1+e_Gamma) ./10e6 ; % Mbps bps = bit/s
e_t = 1000.*e_taskflow./e_r ; % 毫秒 ms

m_NO = -174+10.*log10(m_i.*b)+NF;
m_NO = 10.^(m_NO/10);
m_p_rx = (10.^((p-m_phi)./10)).*(abs(m_h)).^2;
m_Gamma = ( m_p_rx ) ./ (m_NO);
m_r = m_i.*b.*log2(1+m_Gamma) ./10e6 ; % Mbps bps = bit/s

m_t = 1000.*m_taskflow./m_r ; % 毫秒 ms

hold on
total_score_list = zeros(10,1);
for i = 1:60
    total_score_list(i) = t2_e_score(i,e_t,e_r,e_r_LSA);
end
scatter(1:60,total_score_list,"r")
plot(1:60,total_score_list,"r")
xlabel("eMBB切片数")

```



```
ylabel("用户得分")
title("1000ms情况下eMBB切片数与得分的关系")
```

附录 G problem_2_m_score_plot

```
clc,clear,close all

format short g

filePath = fullfile('附件', '附件2', 'channel_data.xlsx');

phi = readmatrix(filePath, 'Sheet', '大规模衰减', "Range", "B2:Q1001"); % U2 e4 m10
U_phi = phi(:,1:2);
e_phi = phi(:,3:6);
m_phi = phi(:,7:16);
h = readmatrix(filePath, 'Sheet', '小规模瑞丽衰减', "Range", "B2:Q1001");
U_h = h(:,1:2);
e_h = h(:,3:6);
m_h = h(:,7:16);
taskflow = readmatrix(filePath, 'Sheet', '用户任务流', "Range", "B2:Q1001");
U_taskflow = taskflow(:,1:2);
e_taskflow = taskflow(:,3:6);
m_taskflow = taskflow(:,7:16);
axis_xy = readmatrix(filePath, 'Sheet', '用户位置', "Range", "B2:AG1001");
U_axis_xy = axis_xy(:,1:4);
e_axis_xy = axis_xy(:,5:12);
m_axis_xy = axis_xy(:,13:32);
p = 30; % dbm
b = 360000; % Hz
U_i = 10; % RB占用量
e_i = 5;
m_i = 2;
NF = 7;
alpha = 0.95;
e_r_LSA = 50; % Mbps

U_NO = -174+10.*log10(U_i.*b)+NF;
U_NO = 10.^(U_NO/10);
U_p_rx = (10.^((p-U_phi)./10)).*(abs(U_h)).^2;
U_Gamma = ( U_p_rx ) ./ (U_NO);
U_r = U_i.*b.*log2(1+U_Gamma) ./10e6 ; % Mbps bps = bit/s

U_t = 1000.*U_taskflow./U_r ; % 毫秒 ms
```

```

e_N0 = -174+10.*log10(e_i.*b)+NF;
e_N0 = 10.^(e_N0/10);
e_p_rx = (10.^(p-e_phi)./10)).*(abs(e_h)).^2;
e_Gamma = ( e_p_rx ) ./ (e_N0);

e_r = e_i.*b.*log2(1+e_Gamma) ./10e6 ; % Mbps bps = bit/s
e_t = 1000.*e_taskflow./e_r ;          % 毫秒 ms

m_N0 = -174+10.*log10(m_i.*b)+NF;
m_N0 = 10.^(m_N0/10);
m_p_rx = (10.^(p-m_phi)./10)).*(abs(m_h)).^2;
m_Gamma = ( m_p_rx ) ./ (m_N0);
m_r = m_i.*b.*log2(1+m_Gamma) ./10e6 ; % Mbps bps = bit/s

m_t = 1000.*m_taskflow./m_r ;          % 毫秒 ms

hold on
total_score_list = zeros(25,1);
for i = 1:50
    total_score_list(i) = t2_m_score(i,m_t);
end
scatter(1:50,total_score_list,"r*")
plot(1:50,total_score_list,"r")
xlabel("mMTC切片数")
ylabel("用户得分")
title("1000ms情况下mMTC切片数与得分的关系")

```

附录 H problem_2_U_score_plot

```

clc,clear,close all

format short g

filePath = fullfile('附件', '附件2', 'channel_data.xlsx');

phi = readmatrix(filePath, 'Sheet', '大规模衰减', "Range", "B2:Q1001"); % U2 e4 m10
U_phi = phi(:,1:2);
e_phi = phi(:,3:6);
m_phi = phi(:,7:16);
h = readmatrix(filePath, 'Sheet', '小规模瑞丽衰减', "Range", "B2:Q1001");
U_h = h(:,1:2);
e_h = h(:,3:6);

```

```

m_h = h(:,7:16);
taskflow = readmatrix(filePath, 'Sheet', '用户任务流', "Range", "B2:Q1001");
U_taskflow = taskflow(:,1:2);
e_taskflow = taskflow(:,3:6);
m_taskflow = taskflow(:,7:16);
axis_xy = readmatrix(filePath, 'Sheet', '用户位置', "Range", "B2:AG1001");
U_axis_xy = axis_xy(:,1:4);
e_axis_xy = axis_xy(:,5:12);
m_axis_xy = axis_xy(:,13:32);
p = 30; % dbm
b = 360000; % Hz
U_i = 10; % RB占用量
e_i = 5;
m_i = 2;
NF = 7;
alpha = 0.95;
r_LSA = 50; % Mbps

U_NO = -174+10.*log10(U_i.*b)+NF;
U_NO = 10.^(U_NO/10);
U_p_rx = (10.^((p-U_phi)./10)).*(abs(U_h)).^2;
U_Gamma = ( U_p_rx ) ./ (U_NO);
U_r = U_i.*b.*log2(1+U_Gamma) ./10e6 ; % Mbps bps = bit/s

U_t = 1000.*U_taskflow./U_r ; % 毫秒 ms

e_NO = -174+10.*log10(e_i.*b)+NF;
e_NO = 10.^(e_NO/10);
e_p_rx = (10.^((p-e_phi)./10)).*(abs(e_h)).^2;
e_Gamma = ( e_p_rx ) ./ (e_NO);

e_r = e_i.*b.*log2(1+e_Gamma) ./10e6 ; % Mbps bps = bit/s
e_t = 1000.*e_taskflow./e_r ; % 毫秒 ms

m_NO = -174+10.*log10(m_i.*b)+NF;
m_NO = 10.^(m_NO/10);
m_p_rx = (10.^((p-m_phi)./10)).*(abs(m_h)).^2;
m_Gamma = ( m_p_rx ) ./ (m_NO);
m_r = m_i.*b.*log2(1+m_Gamma) ./10e6 ; % Mbps bps = bit/s

m_t = 1000.*m_taskflow./m_r ; % 毫秒 ms

hold on

```

```

total_score_list = zeros(5,1)
for i = 1:5
    total_score_list(i) = t2_U_score(i,U_t)
end
scatter(1:5,total_score_list,"r*")
plot(1:5,total_score_list,"r")
xlabel("URLLC切片数")
ylabel("用户得分")
title("1000ms情况下URLLC切片数与得分的关系")

```

附录 I problem_2_result_show

```

clc,clear,close all

U = [10,20,10,10,10,20,10,10,20,10];
e = [15,5,16,15,16,5,16,15,5,16];
m = [25,25,24,25,24,25,24,25,25,24];

hold on
scatter(0:100:900,U,"filled","r")
scatter(0:100:900,e,"filled","b")
scatter(0:100:900,m,"filled","g")
plot(0:100:900,U,"r")
plot(0:100:900,e,"b")
plot(0:100:900,m,"g")
legend("", "", "", "URLLC切片", "eMBB切片", "mMTC切片", "east")

title("第二问动态优化结果图")
xlabel("调度时间（单位是毫秒）")
ylabel("资源块数量")

```

附录 J problem_3

```

clc,clear,close all

format long g

[sum_total,U_count,e_count,m_count] = t3_p_count(10,30,10);

sum_list = zeros(125,1);
p3_list = cell(125,1);

```

```

U_count_list = cell(125,1);
e_count_list = cell(125,1);
m_count_list = cell(125,1);
n = 0;
% for p1 = [10,15,20,25,30]
%     for p2 = [10,15,20,25,30]
%         for p3 = [10,15,20,25,30]
%             n = n + 1; % 101
%             [sum_total,U_count,e_count,m_count] = t3_p_count(p1,p2,p3);
%             sum_list(n) = sum(sum_total);
%             p3_list{n} = [p1,p2,p3];
%             U_count_list{n} = U_count;
%             e_count_list{n} = e_count;
%             m_count_list{n} = m_count;
%         end
%     end
% end

% [sum_total,U_count,e_count,m_count] = t3_p_count(10,10,30)
[sum_total,U_count,e_count,m_count] = t3_p_count(10,30,10)
% [sum_total,U_count,e_count,m_count] = t3_p_count(30,10,10)

```

附录 K problem_3_plot

```

clc,clear,close all

filePath = fullfile('附件', '附件3', 'taskflow.xlsx');

axis_xy = readmatrix(filePath,'Sheet','用户位置',"Range","B2:CS1001");
axis_xy = axis_xy(1,:);

hold on

for i = 1:48 % 6 12 30
    scatter(axis_xy(2*i-1),axis_xy(2*i),15,"r","filled")
end

scatter(0, 500, 50,"b","filled")
scatter(-433.0127, -250, 50,"b","filled")
scatter(433.0127, -250, 50,"b","filled")

labels = {'U1', 'U2', 'U3', 'U4','U5', 'U6','e1', 'e2', 'e3', 'e4','e5', ...
    'e6', 'e7', 'e8','e9', 'e10','e11', 'e12','m1', 'm2', 'm3',
    'm4','m5','m6','m7','m8','m9','m10',...

```

```

    'm11', 'm12', 'm13', 'm14', 'm15', 'm16', 'm17', 'm18', 'm19', 'm20' ...
    'm21', 'm22', 'm23', 'm24', 'm25', 'm26', 'm27', 'm28', 'm29', 'm30'}
labels1 = {'BS1', 'BS2', 'BS3'};

% 在每个点上添加标注
for i = 1:48
    text(axis_xy(2*i-1), axis_xy(2*i), labels{i}, 'VerticalAlignment', 'bottom',
        'HorizontalAlignment', 'right');
end
text(0, 500, labels1{1}, 'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'right');
text(-433.0127, -250, labels1{2}, 'VerticalAlignment', 'bottom', 'HorizontalAlignment',
    'right');
text(433.0127, -250, labels1{3}, 'VerticalAlignment', 'bottom', 'HorizontalAlignment',
    'right');

xlabel("X轴")
ylabel("Y轴")
title("第三问用户和基站位置 (0ms时)")

% BS1: (0, 500)
% BS2: (-433.0127, -250)
% BS3: (433.0127, -250)

```

附录 L problem_3_result_show

```

clc, clear, close all

U1 = [10,20,10,20,20,10,20,10,10,10];
e1 = [15,10,16, 8, 9,15, 7,15,15,16];
m1 = [25,20,24,22,21,25,23,25,25,24];

hold on
scatter(0:100:900,U1,"filled","r")
scatter(0:100:900,e1,"filled","b")
scatter(0:100:900,m1,"filled","g")
plot(0:100:900,U1,"r")
plot(0:100:900,e1,"b")
plot(0:100:900,m1,"g")
legend("", "", "", "URLLC切片", "eMBB切片", "mMTC切片", "east")

title("第三问基站1 (SB1) 动态优化结果图")
xlabel("调度时间 (单位是毫秒)")
ylabel("资源块数量")

%

```

```

% U2 = [20,10,10,10,20,10,10,10,20,10];
% e2 = [5,15,16, 18, 9,15,17,15, 5,16];
% m2 = [25,25,24,22,21,25,23,25,25,24];
%
% hold on
% scatter(0:100:900,U2,"filled","r")
% scatter(0:100:900,e2,"filled","b")
% scatter(0:100:900,m2,"filled","g")
% plot(0:100:900,U2,"r")
% plot(0:100:900,e2,"b")
% plot(0:100:900,m2,"g")
% legend("", "", "", "URLLC切片", "eMBB切片", "mMTC切片", "east")
%
% title("第三问基站2 (SB2) 动态优化结果图")
% xlabel("调度时间 (单位是毫秒) ")
% ylabel("资源块数量")

% U2 = [10,12,10,10,20,10,10,10,20,10];
% e2 = [15,20,16,17, 9,15,16,16,10,16];
% m2 = [25,18,24,23,21,25,24,24,20,24];
%
% hold on
% scatter(0:100:900,U2,"filled","r")
% scatter(0:100:900,e2,"filled","b")
% scatter(0:100:900,m2,"filled","g")
% plot(0:100:900,U2,"r")
% plot(0:100:900,e2,"b")
% plot(0:100:900,m2,"g")
% legend("", "", "", "URLLC切片", "eMBB切片", "mMTC切片", "east")
%
% title("第三问基站3 (SB3) 动态优化结果图")
% xlabel("调度时间 (单位是毫秒) ")
% ylabel("资源块数量")

% U2 = [10,11,10,10,10,10,11,10,10,10];
% e2 = [11,10,10,11,10,11,11,10,11,10];
% m2 = [25,28,29,23,29,25,24,24,25,24];
%
% hold on
% scatter(0:100:900,U2,"r")
% scatter(0:100:900,e2,"b")
% scatter(0:100:900,m2,"g")
% plot(0:100:900,U2,"r")
% plot(0:100:900,e2,"b")
% plot(0:100:900,m2,"g")

```

```
% legend("", "", "", "基站1功率", "基站2功率", "基站3功率", "east")
%
% title("第三问三个微基站发射功率的动态优化结果图")
% xlabel("调度时间 (单位是毫秒) ")
% ylabel("发射功率 (单位是dBm) ")
```

附录 M problem_4

```
clc,clear,close all

format long g

[sum_total,U_count,e_count,m_count,U_total_idx,e_total_idx,m_total_idx] =
    t4_p_count(10,10,30,40);

count_list = [U_total_idx,e_total_idx,m_total_idx]

writematrix(count_list, 't4_MBS_SBS.xlsx');
```

附录 N problem_4_plot

```
clc,clear,close all

filePath = fullfile('附件', '附件4', 'taskflow.xlsx');

axis_xy = readmatrix(filePath,'Sheet','用户位置',"Range","B2:EK1001");
axis_xy = axis_xy(1,:);

hold on

for i = 1:70 % 10 20 40
    scatter(axis_xy(2*i-1),axis_xy(2*i),15,"r","filled")
end

scatter(0, 0, 50,"k","filled")
scatter(0, 500, 50,"b","filled")
scatter(-433.0127, -250, 50,"b","filled")
scatter(433.0127, -250, 50,"b","filled")

labels = {'U1', 'U2', 'U3', 'U4','U5', 'U6','U3', 'U4','U5', 'U6',...
    'e1', 'e2', 'e3', 'e4','e5','e6', 'e7', 'e8','e9', 'e10',...}
```



```

    'e11', 'e12', 'e13', 'e14', 'e15', 'e16', 'e17', 'e18', 'e19', 'e20', ...
    'm1', 'm2', 'm3', 'm4', 'm5', 'm6', 'm7', 'm8', 'm9', 'm10', ...
    'm11', 'm12', 'm13', 'm14', 'm15', 'm16', 'm17', 'm18', 'm19', 'm20' ...
    'm21', 'm22', 'm23', 'm24', 'm25', 'm26', 'm27', 'm28', 'm29', 'm30' ...
    'm31', 'm32', 'm33', 'm34', 'm35', 'm36', 'm37', 'm38', 'm39', 'm40' ...
}
labels1 = {'SBS_1', 'SBS_2', 'SBS_3', 'MBS_1'};

% 在每个点上添加标注
for i = 1:70
    text(axis_xy(2*i-1), axis_xy(2*i), labels{i}, 'VerticalAlignment', 'bottom',
        'HorizontalAlignment', 'right');
end

text(0, 500, labels1{1}, 'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'right');
text(-433.0127, -250, labels1{2}, 'VerticalAlignment', 'bottom', 'HorizontalAlignment',
    'right');
text(433.0127, -250, labels1{3}, 'VerticalAlignment', 'bottom', 'HorizontalAlignment',
    'right');
text(0, 0, labels1{4}, 'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'right');
xlabel("X轴")
ylabel("Y轴")
title("第四问用户和基站位置 (0ms时)")

% MBS_1: (0, 0)
% SBS_1: (0, 500)
% SBS_2: (-433.0127, -250)
% SBS_3: (433.0127, -250)

```

附录 O problem_4_result_show

```

clc, clear, close all

U1 = [10,20,10,20,20,11,20,10,13,13];
e1 = [15, 8,16, 8, 9,15, 7,20,15,16];
m1 = [25,22,24,22,21,24,23,20,22,21];

hold on
scatter(0:100:900,U1,"filled","r")
scatter(0:100:900,e1,"filled","b")
scatter(0:100:900,m1,"filled","g")
plot(0:100:900,U1,"r")
plot(0:100:900,e1,"b")
plot(0:100:900,m1,"g")
legend("", "", "", "URLLC切片", "eMBB切片", "mMTC切片", "east")

```

```

title("第四问微基站1 (SBS1) 动态优化结果图")
xlabel("调度时间 (单位是毫秒) ")
ylabel("资源块数量")

% U2 = [20,10,10,10,20,11,10,13,20,10];
% e2 = [ 5,15,16,15, 9,15,17,16, 9,20];
% m2 = [25,25,24,25,21,24,23,21,21,20];
%
% hold on
% scatter(0:100:900,U2,"filled","r")
% scatter(0:100:900,e2,"filled","b")
% scatter(0:100:900,m2,"filled","g")
% plot(0:100:900,U2,"r")
% plot(0:100:900,e2,"b")
% plot(0:100:900,m2,"g")
% legend("", "", "", "URLLC切片", "eMBB切片", "mMTC切片", "east")
%
% title("第四问微基站2 (SBS2) 动态优化结果图")
% xlabel("调度时间 (单位是毫秒) ")
% ylabel("资源块数量")

% U2 = [10,12,20,10,20,11,10,10,20,10];
% e2 = [15,20, 6,17, 9,15,16,18,10,18];
% m2 = [25,18,24,23,21,24,24,22,20,22];
%
% hold on
% scatter(0:100:900,U2,"filled","r")
% scatter(0:100:900,e2,"filled","b")
% scatter(0:100:900,m2,"filled","g")
% plot(0:100:900,U2,"r")
% plot(0:100:900,e2,"b")
% plot(0:100:900,m2,"g")
% legend("", "", "", "URLLC切片", "eMBB切片", "mMTC切片", "east")
%
% title("第四问微基站3 (SBS3) 动态优化结果图")
% xlabel("调度时间 (单位是毫秒) ")
% ylabel("资源块数量")

% U2 = [10,12,10,10,20,10,10,10,20,10];
% e2 = [15,20,16,17, 9,15,16,16,10,16];
% m2 = [25,18,24,23,21,25,24,24,20,24];
%
% hold on
% scatter(0:100:900,U2,"filled","r")

```

```

% scatter(0:100:900,e2,"filled","b")
% scatter(0:100:900,m2,"filled","g")
% plot(0:100:900,U2,"r")
% plot(0:100:900,e2,"b")
% plot(0:100:900,m2,"g")
% legend("", "", "", "URLLC切片", "eMBB切片", "mMTC切片", "east")
%
% title("第四问宏基站 (MBS_1) 动态优化结果图")
% xlabel("调度时间 (单位是毫秒) ")
% ylabel("资源块数量")

% U2 = [10,11,10,10,10,10,11,10,10,10];
% e2 = [11,10,10,11,10,11,11,10,11,10];
% m2 = [25,28,29,23,29,25,24,24,25,24];
% m22 = [36,35,39,38,36,37,37,34,38,39];
%
% hold on
% scatter(0:100:900,U2,"r")
% scatter(0:100:900,e2,"b")
% scatter(0:100:900,m2,"g")
% scatter(0:100:900,m22,"k")
% plot(0:100:900,U2,"r")
% plot(0:100:900,e2,"b")
% plot(0:100:900,m2,"g")
% plot(0:100:900,m22,"k")
% legend("", "", "", "", "微基站1功率", "微基站2功率", "微基站3功率", "宏基站1功率", "east")
%
% title("第四问3个微基站和1个宏基站发射功率的动态优化结果图")
% xlabel("调度时间 (单位是毫秒) ")
% ylabel("发射功率 (单位是dBm) ")

```

附录 P problem_5

```

clc,clear,close all

format long g

p_static = 28.*4; % J
delta = 0.75;
eta = 0.35;+
N_active = [50,50,50,100];

p_rb = delta.*sum(N_active);
p_transmit = ( 10.^( ([10,10,30,40]-30)./10 ) )./1000; % 要转化单位

```

```
p_tx = sum(p_transmit)./eta;

p = p_static + p_rb + p_tx % 单位 J
```

附录 Q problem_5_result_show

```
clc, clear, close all

U1 = [10,21,10,20,20,11,20,10,13,13];
e1 = [15, 7,16, 8, 9,15, 7,20,15,16];
m1 = [25,22,24,22,21,24,23,20,22,21];

hold on
scatter(0:100:900,U1,"filled","r")
scatter(0:100:900,e1,"filled","b")
scatter(0:100:900,m1,"filled","g")
plot(0:100:900,U1,"r")
plot(0:100:900,e1,"b")
plot(0:100:900,m1,"g")
legend("", "", "", "URLLC切片", "eMBB切片", "mMTC切片", "east")

title("第五问微基站1 (SBS1) 动态优化结果图")
xlabel("调度时间 (单位是毫秒) ")
ylabel("资源块数量")
%
%
% U2 = [20,11,10,11,20,11,10,13,20,10];
% e2 = [ 5,15,16,15, 9,16,17,16, 9,20];
% m2 = [25,24,24,24,21,23,23,21,21,20];
%
% hold on
% scatter(0:100:900,U2,"filled","r")
% scatter(0:100:900,e2,"filled","b")
% scatter(0:100:900,m2,"filled","g")
% plot(0:100:900,U2,"r")
% plot(0:100:900,e2,"b")
% plot(0:100:900,m2,"g")
% legend("", "", "", "URLLC切片", "eMBB切片", "mMTC切片", "east")
%
% title("第五问微基站2 (SBS2) 动态优化结果图")
% xlabel("调度时间 (单位是毫秒) ")
% ylabel("资源块数量")

% U2 = [10,12,20,11,20,11,10,10,20,11];
% e2 = [15,20, 6,16, 9,15,16,18,10,18];
```

```

% m2 = [25,18,24,23,21,24,24,22,20,21];
%
% hold on
% scatter(0:100:900,U2,"filled","r")
% scatter(0:100:900,e2,"filled","b")
% scatter(0:100:900,m2,"filled","g")
% plot(0:100:900,U2,"r")
% plot(0:100:900,e2,"b")
% plot(0:100:900,m2,"g")
% legend("", "", "", "URLLC切片", "eMBB切片", "mMTC切片", "east")
%
% title("第五问微基站3 (SBS3) 动态优化结果图")
% xlabel("调度时间 (单位是毫秒) ")
% ylabel("资源块数量")

% U2 = [10,11,10,10,20,10,11,10,20,10];
% e2 = [15,20,16,17,10,15,16,16,10,16];
% m2 = [25,19,24,23,20,25,25,24,20,24];
%
% hold on
% scatter(0:100:900,U2,"filled","r")
% scatter(0:100:900,e2,"filled","b")
% scatter(0:100:900,m2,"filled","g")
% plot(0:100:900,U2,"r")
% plot(0:100:900,e2,"b")
% plot(0:100:900,m2,"g")
% legend("", "", "", "URLLC切片", "eMBB切片", "mMTC切片", "east")
%
% title("第五问宏基站 (MBS_1) 动态优化结果图")
% xlabel("调度时间 (单位是毫秒) ")
% ylabel("资源块数量")

% U2 = [10,11,10,10,10,10,11,10,10,10];
% e2 = [11,10,10,11,10,11,11,10,11,10];
% m2 = [16,15,17,19,18,17,18,16,16,18];
% m22 = [20,19,21,22,19,18,21,22,22,21];
%
% hold on
% scatter(0:100:900,U2,"r")
% scatter(0:100:900,e2,"b")
% scatter(0:100:900,m2,"g")
% scatter(0:100:900,m22,"k")
% plot(0:100:900,U2,"r")
% plot(0:100:900,e2,"b")
% plot(0:100:900,m2,"g")

```

```

% plot(0:100:900,m22,"k")
% legend("", "", "", "", "微基站1功率", "微基站2功率", "微基站3功率", "宏基站1功率", "east")
%
% title("第五问3个微基站和1个宏基站发射功率的动态优化结果图")
% xlabel("调度时间 (单位是毫秒) ")
% ylabel("发射功率 (单位是dBm) ")

```

附录 R t2_e_score

```

function [total_score,L_list] = t2_e_score(num,e_t,e_r,e_r_LSA)

U_t = e_t;
num_flag = zeros(num,1);
L_list = zeros(1000,4);
chengfadian = {};
n = 0;
for i = 1:size(U_t,1)
    for j = 1:size(U_t,2)
        [iii,ii] = min(num_flag);
        Q = ceil(num_flag(ii));
        num_flag(ii) = U_t(i,j) + Q;
        L_list(i,j) = num_flag(ii);
        if any(num_flag(ii) > 100) && U_t(i,j) ~= 0
            % fprintf("第%d毫秒,用户%d受到惩罚,Q为%d,总用时L为%d\n",i,j,Q,num_flag(ii));
            n = n + 1;
            chengfadian{n} = [i,j];
        end
    end
end
if any(num_flag > 0)
    [iiii] = find(num_flag > 0);
    num_flag(iiii) = num_flag(iiii) - 1;
end
end
score_list = e_r./e_r_LSA;
for i = chengfadian
    i = cell2mat(i);
    score_list(i(1),i(2)) = -3;
end
total_score = sum(score_list,"all");

end

```

附录 S t2_e_score_100ms

```
function [total_score,L_list] = t2_e_score_100ms(k,num,e_t,e_r,e_r_LSA)

U_t = e_t;
num_flag = zeros(num,1);
L_list = zeros(100,4);
chengfadian = {};
n = 0;
for i = 1:size(U_t,1)
    for j = 1:size(U_t,2)
        [iii,ii] = min(num_flag);
        Q = ceil(num_flag(ii));
        num_flag(ii) = U_t(i,j) + Q;
        L_list(i,j) = num_flag(ii);
        if any(num_flag(ii) > 100) && U_t(i,j) ~= 0
            % fprintf("第%d毫秒,用户%d受到惩罚,Q为%d,总用时L为%d\n",i,j,Q,num_flag(ii));
            n = n + 1;
            chengfadian{n} = [i,j];
        end
    end
end
if any(num_flag > 0)
    [iiii] = find(num_flag > 0);
    num_flag(iiii) = num_flag(iiii) - 1;
end
end
score_list = e_r./e_r_LSA;
for i = chengfadian
    i = cell2mat(i);
    score_list(i(1),i(2)) = -3;
end
total_score = sum(score_list,"all");

end
```

附录 T t2_m_score

```
function [total_score,L_list] = t2_m_score(num,m_t)

U_t = m_t;
num_flag = zeros(num,1);
Q_list = zeros(1000,10);
L_list = zeros(1000,10);
chengfadian = {};
```

```

n = 0;
for i = 1:size(U_t,1)
    for j = 1:size(U_t,2)
        [iii,ii] = min(num_flag);
        Q = ceil(num_flag(ii));
        num_flag(ii) = U_t(i,j) + Q;
        Q_list(i,j) = Q;
        L_list(i,j) = num_flag(ii);
        if any(num_flag(ii) > 500) && U_t(i,j) ~= 0
            % fprintf("第%d毫秒,用户%d受到惩罚,Q为%d,总用时L为%d\n",i,j,Q,num_flag(ii));
            n = n + 1;
            chengfadian{n} = [i,j];
        end
    end
    if any(num_flag > 0)
        [iiii] = find(num_flag > 0);
        num_flag(iiii) = num_flag(iiii) - 1;
    end
end
score_list = sum(Q_list <= 100)./10000; % Q_list
for i = chengfadian
    i = cell2mat(i);
    score_list(i(1),i(2)) = -1;
end
total_score = sum(score_list,"all");

end

```

附录 U t2_m_score_100ms

```

function [total_score,L_list] = t2_m_score_100ms(num,m_t)

U_t = m_t;
num_flag = zeros(num,1);
Q_list = zeros(100,10);
L_list = zeros(100,10);
chengfadian = {};
n = 0;
for i = 1:size(U_t,1)
    for j = 1:size(U_t,2)
        [iii,ii] = min(num_flag);
        Q = ceil(num_flag(ii));
        num_flag(ii) = U_t(i,j) + Q;
        Q_list(i,j) = Q;
        L_list(i,j) = num_flag(ii);
    end
end

```



```

        if any(num_flag(ii) > 500) && U_t(i,j) ~= 0
            % fprintf("第%d毫秒,用户%d受到惩罚,Q为%d,总用时L为%d\n",i,j,Q,num_flag(ii));
            n = n + 1;
            chengfadian{n} = [i,j];
        end
    end
    if any(num_flag > 0)
        [iiii] = find(num_flag > 0);
        num_flag(iiii) = num_flag(iiii) - 1;
    end
end
score_list = sum(Q_list <= 100)./10000; % Q_list
for i = chengfadian
    i = cell2mat(i);
    score_list(i(1),i(2)) = -1;
end
total_score = sum(score_list,"all");

end

```

附录 V t2_U_score

```

function [total_score,L_list] = t2_U_score(num,U_t)

num_flag = zeros(num,1);
L_list = zeros(1000,2);
for i = 1:size(U_t,1)
    for j = 1:size(U_t,2)
        [iii,ii] = min(num_flag);
        Q = ceil(num_flag(ii));
        num_flag(ii) = U_t(i,j) + ceil(num_flag(ii));
        L_list(i,j) = num_flag(ii);
        if any(num_flag(ii) > 5) && U_t(i,j) ~= 0
            % fprintf("第%d毫秒,用户%d受到惩罚,Q为%d毫秒,总用时L为%.6f毫秒\n",i,j,Q,num_flag(ii));
        end
    end
    if any(num_flag > 0)
        [iiii] = find(num_flag > 0);
        num_flag(iiii) = num_flag(iiii) - 1;
    end
end
score_list = 0.95.^L_list;
score_list(score_list < 0.95^5) = -5;
total_score = sum(score_list,"all");

```

```
end
```

附录 W t2_U_score_100ms

```
function [total_score,L_list] = t2_U_score_100ms(k,num,U_t)

U_t = U_t;

num_flag = zeros(num,1);
L_list = zeros(100,2);
for i = 1:100
    for j = 1:size(U_t,2)
        [iii,ii] = min(num_flag);
        Q = ceil(num_flag(ii));
        num_flag(ii) = U_t(i,j) + ceil(num_flag(ii));
        L_list(i,j) = num_flag(ii);
        if any(num_flag(ii) > 5) && U_t(i,j) ~= 0
            %
            fprintf("第%d毫秒,用户%d受到惩罚,Q为%d毫秒,总用时L为%.6f毫秒\n",i,j,Q,num_flag(ii));
        end
    end
    if any(num_flag > 0)
        [iiii] = find(num_flag > 0);
        num_flag(iiii) = num_flag(iiii) - 1;
    end
end

L_list;
score_list = 0.95.^L_list;
score_list(score_list < 0.95^5) = -5;
total_score = sum(score_list,"all");

end
```

附录 X t3_p_count

```
function [sum_total,U_count,e_count,m_count] = t3_p_count(p1,p2,p3)

format long g

filePathbs1 = fullfile('附件', '附件3', 'BS1.xlsx');
```

```

filePathbs2 = fullfile('附件', '附件3', 'BS2.xlsx');
filePathbs3 = fullfile('附件', '附件3', 'BS3.xlsx');
filePath = fullfile('附件', '附件3', 'taskflow.xlsx');

phibs1 = readmatrix(filePathbs1, 'Sheet', '大规模衰减', "Range", "B2:AW1001"); % U2 e4 m10
U_phibs1 = phibs1(:,1:6);
e_phibs1 = phibs1(:,7:18);
m_phibs1 = phibs1(:,19:48);

phibs2 = readmatrix(filePathbs2, 'Sheet', '大规模衰减', "Range", "B2:AW1001"); % U2 e4 m10
U_phibs2 = phibs2(:,1:6);
e_phibs2 = phibs2(:,7:18);
m_phibs2 = phibs2(:,19:48);

phibs3 = readmatrix(filePathbs3, 'Sheet', '大规模衰减', "Range", "B2:AW1001"); % U2 e4 m10
U_phibs3 = phibs3(:,1:6);
e_phibs3 = phibs3(:,7:18);
m_phibs3 = phibs3(:,19:48);

h1 = readmatrix(filePathbs1, 'Sheet', '小规模瑞丽衰减', "Range", "B2:AW1001"); % U2 e4 m10
U_h1 = h1(:,1:6);
e_h1 = h1(:,7:18);
m_h1 = h1(:,19:48);

h2 = readmatrix(filePathbs2, 'Sheet', '小规模瑞丽衰减', "Range", "B2:AW1001"); % U2 e4 m10
U_h2 = h2(:,1:6);
e_h2 = h2(:,7:18);
m_h2 = h2(:,19:48);

h3 = readmatrix(filePathbs3, 'Sheet', '小规模瑞丽衰减', "Range", "B2:AW1001"); % U2 e4 m10
U_h3 = h3(:,1:6);
e_h3 = h3(:,7:18);
m_h3 = h3(:,19:48);

taskflow = readmatrix(filePath, 'Sheet', '用户任务流', "Range", "B2:AW1001");
U_taskflow = taskflow(:,1:6);
e_taskflow = taskflow(:,7:18);
m_taskflow = taskflow(:,19:48);

axis_xy = readmatrix(filePath, 'Sheet', '用户位置', "Range", "B2:CS1001");
U_axis_xy = axis_xy(:,1:12); % 6
e_axis_xy = axis_xy(:,13:36); % 12
m_axis_xy = axis_xy(:,37:96); % 30

% p1 = 10; % dbm

```

```

% p2 = 10;
% p3 = 10;
b = 360000; % Hz
U_i = 10; % RB占用量
e_i = 5;
m_i = 2;
NF = 7;
alpha = 0.95;
e_r_LSA = 50; % Mbps

U_N0 = -174+10.*log10(U_i.*b)+NF;
U_N0 = 10.^(U_N0/10);

U1_p_rx = (10.^((p1-U_phibs1)./10)).*(abs(U_h1)).^2;
U2_p_rx = (10.^((p2-U_phibs2)./10)).*(abs(U_h2)).^2;
U3_p_rx = (10.^((p3-U_phibs3)./10)).*(abs(U_h3)).^2;

U1_Gamma = ( U1_p_rx ) ./ (U2_p_rx + U3_p_rx + U_N0);
U2_Gamma = ( U2_p_rx ) ./ (U1_p_rx + U3_p_rx + U_N0);
U3_Gamma = ( U3_p_rx ) ./ (U1_p_rx + U2_p_rx + U_N0);
U1_Gamma1 = ( U1_p_rx ) ./ (U_N0);
U2_Gamma1 = ( U2_p_rx ) ./ (U_N0);
U3_Gamma1 = ( U3_p_rx ) ./ (U_N0);

U1_r = U_i.*b.*log2(1+U1_Gamma) ./10e6 ; % Mbps bps = bit/s
U2_r = U_i.*b.*log2(1+U2_Gamma) ./10e6 ;
U3_r = U_i.*b.*log2(1+U3_Gamma) ./10e6 ;
U1_r1 = U_i.*b.*log2(1+U1_Gamma1) ./10e6 ; % Mbps bps = bit/s
U2_r1 = U_i.*b.*log2(1+U2_Gamma1) ./10e6 ;
U3_r1 = U_i.*b.*log2(1+U3_Gamma1) ./10e6 ;

U1_t = 1000.*U_taskflow./U1_r ; % 毫秒 ms
U2_t = 1000.*U_taskflow./U2_r ;
U3_t = 1000.*U_taskflow./U3_r ;
U1_t1 = 1000.*U_taskflow./U1_r1 ; % 毫秒 ms
U2_t1 = 1000.*U_taskflow./U2_r1 ;
U3_t1 = 1000.*U_taskflow./U3_r1 ;

e_N0 = -174+10.*log10(e_i.*b)+NF;
e_N0 = 10.^(e_N0/10);

e1_p_rx = (10.^((p1-e_phibs1)./10)).*(abs(e_h1)).^2;
e2_p_rx = (10.^((p2-e_phibs2)./10)).*(abs(e_h2)).^2;
e3_p_rx = (10.^((p3-e_phibs3)./10)).*(abs(e_h3)).^2;

```

```

e1_Gamma = ( e1_p_rx ) ./ ( e2_p_rx + e3_p_rx + e_N0);
e2_Gamma = ( e2_p_rx ) ./ ( e1_p_rx + e3_p_rx + e_N0);
e3_Gamma = ( e3_p_rx ) ./ ( e1_p_rx + e2_p_rx + e_N0);
e1_Gamma1 = ( e1_p_rx ) ./ ( e_N0);
e2_Gamma1 = ( e2_p_rx ) ./ ( e_N0);
e3_Gamma1 = ( e3_p_rx ) ./ ( e_N0);

e1_r = U_i.*b.*log2(1+e1_Gamma) ./10e6 ; % Mbps bps = bit/s
e2_r = U_i.*b.*log2(1+e2_Gamma) ./10e6 ;
e3_r = U_i.*b.*log2(1+e3_Gamma) ./10e6 ;
e1_r1 = U_i.*b.*log2(1+e1_Gamma1) ./10e6 ; % Mbps bps = bit/s
e2_r1 = U_i.*b.*log2(1+e2_Gamma1) ./10e6 ;
e3_r1 = U_i.*b.*log2(1+e3_Gamma1) ./10e6 ;

e1_t = 1000.*e_taskflow./e1_r ; % 毫秒 ms
e2_t = 1000.*e_taskflow./e2_r ;
e3_t = 1000.*e_taskflow./e3_r ;
e1_t1 = 1000.*e_taskflow./e1_r1 ; % 毫秒 ms
e2_t1 = 1000.*e_taskflow./e2_r1 ;
e3_t1 = 1000.*e_taskflow./e3_r1 ;

m_N0 = -174+10.*log10(m_i.*b)+NF;
m_N0 = 10.^(m_N0/10);

m1_p_rx = (10.^((p1-m_phibs1)./10)).*(abs(m_h1)).^2;
m2_p_rx = (10.^((p2-m_phibs2)./10)).*(abs(m_h2)).^2;
m3_p_rx = (10.^((p3-m_phibs3)./10)).*(abs(m_h3)).^2;

m1_Gamma = ( m1_p_rx ) ./ ( m2_p_rx + m3_p_rx + m_N0);
m2_Gamma = ( m2_p_rx ) ./ ( m1_p_rx + m3_p_rx + m_N0);
m3_Gamma = ( m3_p_rx ) ./ ( m1_p_rx + m2_p_rx + m_N0);
m1_Gamma1 = ( m1_p_rx ) ./ ( m_N0);
m2_Gamma1 = ( m2_p_rx ) ./ ( m_N0);
m3_Gamma1 = ( m3_p_rx ) ./ ( m_N0);

m1_r = U_i.*b.*log2(1+m1_Gamma) ./10e6 ; % Mbps bps = bit/s
m2_r = U_i.*b.*log2(1+m2_Gamma) ./10e6 ;
m3_r = U_i.*b.*log2(1+m3_Gamma) ./10e6 ;
m1_r1 = U_i.*b.*log2(1+m1_Gamma1) ./10e6 ; % Mbps bps = bit/s
m2_r1 = U_i.*b.*log2(1+m2_Gamma1) ./10e6 ;
m3_r1 = U_i.*b.*log2(1+m3_Gamma1) ./10e6 ;

m1_t = 1000.*m_taskflow./m1_r ; % 毫秒 ms
m2_t = 1000.*m_taskflow./m2_r ;
m3_t = 1000.*m_taskflow./m3_r ;
m1_t1 = 1000.*m_taskflow./m1_r1 ; % 毫秒 ms

```

```

m2_t1 = 1000.*m_taskflow./m2_r1 ;
m3_t1 = 1000.*m_taskflow./m3_r1 ;

[U_total, U_total_idx] = min(cat(3, U1_t, U2_t, U3_t), [], 3);
[U_total1, U_total1_idx] = min(cat(3, U1_t1, U2_t1, U3_t1), [], 3);
[e_total, e_total_idx] = min(cat(3, e1_t, e2_t, e3_t), [], 3);
[e_total1, e_total1_idx] = min(cat(3, e1_t1, e2_t1, e3_t1), [], 3);
[m_total, m_total_idx] = min(cat(3, m1_t, m2_t, m3_t), [], 3);
[m_total1, m_total1_idx] = min(cat(3, m1_t1, m2_t1, m3_t1), [], 3);
uu = (U_total-U_total1)./U_total1;
uu(isnan(uu)) = 0;
ee = (e_total-e_total1)./e_total1;
ee(isnan(ee)) = 0;
mm = (m_total-m_total1)./m_total1;
mm(isnan(mm)) = 0;
sum_total = [sum(uu,"all"),sum(ee,"all"),sum(mm,"all")]./[6,12,30];

U_total_idxcount_1 = sum(U_total_idx(:) == 1);
U_total_idxcount_2 = sum(U_total_idx(:) == 2);
U_total_idxcount_3 = sum(U_total_idx(:) == 3);
U_total1_idxcount_1 = sum(U_total1_idx(:) == 1);
U_total1_idxcount_2 = sum(U_total1_idx(:) == 2);
U_total1_idxcount_3 = sum(U_total1_idx(:) == 3);
U_count = [U_total_idxcount_1,U_total_idxcount_2,U_total_idxcount_3;
    U_total1_idxcount_1,U_total1_idxcount_2,U_total1_idxcount_3];

e_total_idxcount_1 = sum(e_total_idx(:) == 1);
e_total_idxcount_2 = sum(e_total_idx(:) == 2);
e_total_idxcount_3 = sum(e_total_idx(:) == 3);
e_total1_idxcount_1 = sum(e_total1_idx(:) == 1);
e_total1_idxcount_2 = sum(e_total1_idx(:) == 2);
e_total1_idxcount_3 = sum(e_total1_idx(:) == 3);
e_count = [e_total_idxcount_1,e_total_idxcount_2,e_total_idxcount_3;
    e_total1_idxcount_1,e_total1_idxcount_2,e_total1_idxcount_3];

m_total_idxcount_1 = sum(m_total_idx(:) == 1);
m_total_idxcount_2 = sum(m_total_idx(:) == 2);
m_total_idxcount_3 = sum(m_total_idx(:) == 3);
m_total1_idxcount_1 = sum(m_total1_idx(:) == 1);
m_total1_idxcount_2 = sum(m_total1_idx(:) == 2);
m_total1_idxcount_3 = sum(m_total1_idx(:) == 3);
m_count = [m_total_idxcount_1,m_total_idxcount_2,m_total_idxcount_3;
    m_total1_idxcount_1,m_total1_idxcount_2,m_total1_idxcount_3];

end

```

附录 Y t4_p_count

```
function [sum_total,U_count,e_count,m_count,U_total_idx,e_total_idx,m_total_idx] =  
    t4_p_count(p1,p2,p3,pmbs)  
  
format long g  
  
filePathmbs = fullfile('附件', '附件4', 'MBS_1.xlsx');  
filePathsbs_1 = fullfile('附件', '附件4', 'SBS_1.xlsx');  
filePathsbs_2 = fullfile('附件', '附件4', 'SBS_2.xlsx');  
filePathsbs_3 = fullfile('附件', '附件4', 'SBS_3.xlsx');  
filePath = fullfile('附件', '附件4', 'taskflow.xlsx');  
  
phimbs = readmatrix(filePathsbs_1,'Sheet','大规模衰减','Range',"B2:BS1001"); % U2 e4 m10  
U_phimbs = phimbs(:,1:10);  
e_phimbs = phimbs(:,11:30);  
m_phimbs = phimbs(:,31:70);  
  
phibs1 = readmatrix(filePathsbs_1,'Sheet','大规模衰减','Range',"B2:BS1001"); % U2 e4 m10  
U_phibs1 = phibs1(:,1:10);  
e_phibs1 = phibs1(:,11:30);  
m_phibs1 = phibs1(:,31:70);  
  
phibs2 = readmatrix(filePathsbs_2,'Sheet','大规模衰减','Range',"B2:BS1001"); % U2 e4 m10  
U_phibs2 = phibs2(:,1:10);  
e_phibs2 = phibs2(:,11:30);  
m_phibs2 = phibs2(:,31:70);  
  
phibs3 = readmatrix(filePathsbs_3,'Sheet','大规模衰减','Range',"B2:BS1001"); % U2 e4 m10  
U_phibs3 = phibs3(:,1:10);  
e_phibs3 = phibs3(:,11:30);  
m_phibs3 = phibs3(:,31:70);  
  
hmbs = readmatrix(filePathsbs_1,'Sheet','小规模瑞丽衰减','Range',"B2:BS1001"); % U2 e4 m10  
U_hmbs = hmbs(:,1:10);  
e_hmbs = hmbs(:,11:30);  
m_hmbs = hmbs(:,31:70);  
  
h1 = readmatrix(filePathsbs_1,'Sheet','小规模瑞丽衰减','Range',"B2:BS1001"); % U2 e4 m10  
U_h1 = h1(:,1:10);  
e_h1 = h1(:,11:30);  
m_h1 = h1(:,31:70);  
  
h2 = readmatrix(filePathsbs_2,'Sheet','小规模瑞丽衰减','Range',"B2:BS1001"); % U2 e4 m10  
U_h2 = h2(:,1:10);  
e_h2 = h2(:,11:30);
```

```

m_h2 = h2(:,31:70);

h3 = readmatrix(filePathsbs_3,'Sheet','小规模瑞丽衰减','Range',"B2:BS1001"); % U2 e4 m10
U_h3 = h3(:,1:10);
e_h3 = h3(:,11:30);
m_h3 = h3(:,31:70);

taskflow = readmatrix(filePath,'Sheet','用户任务流','Range',"B2:BS1001");
U_taskflow = taskflow(:,1:10);
e_taskflow = taskflow(:,11:30);
m_taskflow = taskflow(:,31:70);

axis_xy = readmatrix(filePath,'Sheet','用户位置','Range',"B2:EK1001");
U_axis_xy = axis_xy(:,1:20); % 10
e_axis_xy = axis_xy(:,21:60); % 20
m_axis_xy = axis_xy(:,61:140); % 40

pmbs = 10;
p1 = 10; % dbm
p2 = 10;
p3 = 10;
b = 360000; % Hz
U_i = 10; % RB占用量
e_i = 5;
m_i = 2;
NF = 7;
alpha = 0.95;
e_r_LSA = 50; % Mbps

U_N0 = -174+10.*log10(U_i.*b)+NF;
U_N0 = 10.^(U_N0/10);

Umbs_p_rx = (10.^((pmbs-U_phimbs)./10)).*(abs(U_hmbs)).^2;
U1_p_rx = (10.^((p1-U_phibs1)./10)).*(abs(U_h1)).^2;
U2_p_rx = (10.^((p2-U_phibs2)./10)).*(abs(U_h2)).^2;
U3_p_rx = (10.^((p3-U_phibs3)./10)).*(abs(U_h3)).^2;

Umbs_Gamma = ( Umbs_p_rx ) ./ (U_N0);
U1_Gamma = ( U1_p_rx ) ./ (U2_p_rx + U3_p_rx + U_N0);
U2_Gamma = ( U2_p_rx ) ./ (U1_p_rx + U3_p_rx + U_N0);
U3_Gamma = ( U3_p_rx ) ./ (U1_p_rx + U2_p_rx + U_N0);
U1_Gamma1 = ( U1_p_rx ) ./ (U_N0);
U2_Gamma1 = ( U2_p_rx ) ./ (U_N0);
U3_Gamma1 = ( U3_p_rx ) ./ (U_N0);

```



```

Umbs_r = U_i.*b.*log2(1+Umbs_Gamma) ./10e6 ;
U1_r = U_i.*b.*log2(1+U1_Gamma) ./10e6 ; % Mbps bps = bit/s
U2_r = U_i.*b.*log2(1+U2_Gamma) ./10e6 ;
U3_r = U_i.*b.*log2(1+U3_Gamma) ./10e6 ;
U1_r1 = U_i.*b.*log2(1+U1_Gamma1) ./10e6 ; % Mbps bps = bit/s
U2_r1 = U_i.*b.*log2(1+U2_Gamma1) ./10e6 ;
U3_r1 = U_i.*b.*log2(1+U3_Gamma1) ./10e6 ;

Umbs_t = 1000.*U_taskflow./Umbs_r ;
U1_t = 1000.*U_taskflow./U1_r ; % 毫秒 ms
U2_t = 1000.*U_taskflow./U2_r ;
U3_t = 1000.*U_taskflow./U3_r ;
U1_t1 = 1000.*U_taskflow./U1_r1 ; % 毫秒 ms
U2_t1 = 1000.*U_taskflow./U2_r1 ;
U3_t1 = 1000.*U_taskflow./U3_r1 ;

e_N0 = -174+10.*log10(e_i.*b)+NF;
e_N0 = 10.^(e_N0/10);

embs_p_rx = (10.^((pmbs-e_phimbs)./10)).*(abs(e_hmbs)).^2;
e1_p_rx = (10.^((p1-e_phibs1)./10)).*(abs(e_h1)).^2;
e2_p_rx = (10.^((p2-e_phibs2)./10)).*(abs(e_h2)).^2;
e3_p_rx = (10.^((p3-e_phibs3)./10)).*(abs(e_h3)).^2;

embs_Gamma = ( embs_p_rx ) ./ (e_N0);
e1_Gamma = ( e1_p_rx ) ./ (e2_p_rx + e3_p_rx + e_N0);
e2_Gamma = ( e2_p_rx ) ./ (e1_p_rx + e3_p_rx + e_N0);
e3_Gamma = ( e3_p_rx ) ./ (e1_p_rx + e2_p_rx + e_N0);
e1_Gamma1 = ( e1_p_rx ) ./ (e_N0);
e2_Gamma1 = ( e2_p_rx ) ./ (e_N0);
e3_Gamma1 = ( e3_p_rx ) ./ (e_N0);

embs_r = U_i.*b.*log2(1+embs_Gamma) ./10e6 ;
e1_r = U_i.*b.*log2(1+e1_Gamma) ./10e6 ; % Mbps bps = bit/s
e2_r = U_i.*b.*log2(1+e2_Gamma) ./10e6 ;
e3_r = U_i.*b.*log2(1+e3_Gamma) ./10e6 ;
e1_r1 = U_i.*b.*log2(1+e1_Gamma1) ./10e6 ; % Mbps bps = bit/s
e2_r1 = U_i.*b.*log2(1+e2_Gamma1) ./10e6 ;
e3_r1 = U_i.*b.*log2(1+e3_Gamma1) ./10e6 ;

embs_t = 1000.*e_taskflow./embs_r ;
e1_t = 1000.*e_taskflow./e1_r ; % 毫秒 ms
e2_t = 1000.*e_taskflow./e2_r ;
e3_t = 1000.*e_taskflow./e3_r ;
e1_t1 = 1000.*e_taskflow./e1_r1 ; % 毫秒 ms
e2_t1 = 1000.*e_taskflow./e2_r1 ;

```

```

e3_t1 = 1000.*e_taskflow./e3_r1 ;

m_NO = -174+10.*log10(m_i.*b)+NF;
m_NO = 10.^(m_NO/10);

mmbs_p_rx = (10.^((pmbs-m_phimbs)./10)).*(abs(m_hmbs)).^2;
m1_p_rx = (10.^((p1-m_phibs1)./10)).*(abs(m_h1)).^2;
m2_p_rx = (10.^((p2-m_phibs2)./10)).*(abs(m_h2)).^2;
m3_p_rx = (10.^((p3-m_phibs3)./10)).*(abs(m_h3)).^2;

mmbs_Gamma = ( mmbs_p_rx ) ./ (m_NO);
m1_Gamma = ( m1_p_rx ) ./ (m2_p_rx + m3_p_rx + m_NO);
m2_Gamma = ( m2_p_rx ) ./ (m1_p_rx + m3_p_rx + m_NO);
m3_Gamma = ( m3_p_rx ) ./ (m1_p_rx + m2_p_rx + m_NO);
m1_Gamma1 = ( m1_p_rx ) ./ (m_NO);
m2_Gamma1 = ( m2_p_rx ) ./ (m_NO);
m3_Gamma1 = ( m3_p_rx ) ./ (m_NO);

mmbs_r = U_i.*b.*log2(1+mmbs_Gamma) ./10e6 ;
m1_r = U_i.*b.*log2(1+m1_Gamma) ./10e6 ; % Mbps bps = bit/s
m2_r = U_i.*b.*log2(1+m2_Gamma) ./10e6 ;
m3_r = U_i.*b.*log2(1+m3_Gamma) ./10e6 ;
m1_r1 = U_i.*b.*log2(1+m1_Gamma1) ./10e6 ; % Mbps bps = bit/s
m2_r1 = U_i.*b.*log2(1+m2_Gamma1) ./10e6 ;
m3_r1 = U_i.*b.*log2(1+m3_Gamma1) ./10e6 ;

mmbs_t = 1000.*m_taskflow./mmbs_r ;
m1_t = 1000.*m_taskflow./m1_r ; % 毫秒 ms
m2_t = 1000.*m_taskflow./m2_r ;
m3_t = 1000.*m_taskflow./m3_r ;
m1_t1 = 1000.*m_taskflow./m1_r1 ; % 毫秒 ms
m2_t1 = 1000.*m_taskflow./m2_r1 ;
m3_t1 = 1000.*m_taskflow./m3_r1 ;

[U_total, U_total_idx] = min(cat(3, U1_t, U2_t, U3_t, Umbs_t), [], 3);
[U_total1, U_total1_idx] = min(cat(3, U1_t1, U2_t1, U3_t1, Umbs_t), [], 3);
[e_total, e_total_idx] = min(cat(3, e1_t, e2_t, e3_t, embs_t), [], 3);
[e_total1, e_total1_idx] = min(cat(3, e1_t1, e2_t1, e3_t1, embs_t), [], 3);
[m_total, m_total_idx] = min(cat(3, m1_t, m2_t, m3_t, mmbs_t), [], 3);
[m_total1, m_total1_idx] = min(cat(3, m1_t1, m2_t1, m3_t1, mmbs_t), [], 3);
uu = (U_total-U_total1)./U_total1;
uu(isnan(uu)) = 0;
ee = (e_total-e_total1)./e_total1;
ee(isnan(ee)) = 0;
mm = (m_total-m_total1)./m_total1;

```

```

mm(isnan(mm)) = 0;

sum_total = [sum(uu,"all"),sum(ee,"all"),sum(mm,"all")]./[6,12,30];

U_total_idxcount_1 = sum(U_total_idx(:) == 1);
U_total_idxcount_2 = sum(U_total_idx(:) == 2);
U_total_idxcount_3 = sum(U_total_idx(:) == 3);
U_total_idxcount_4 = sum(U_total_idx(:) == 4);
U_total1_idxcount_1 = sum(U_total1_idx(:) == 1);
U_total1_idxcount_2 = sum(U_total1_idx(:) == 2);
U_total1_idxcount_3 = sum(U_total1_idx(:) == 3);
U_total1_idxcount_4 = sum(U_total1_idx(:) == 4);
U_count = [U_total_idxcount_1,U_total_idxcount_2,U_total_idxcount_3,U_total_idxcount_4;
           U_total1_idxcount_1,U_total1_idxcount_2,U_total1_idxcount_3,U_total1_idxcount_4];

e_total_idxcount_1 = sum(e_total_idx(:) == 1);
e_total_idxcount_2 = sum(e_total_idx(:) == 2);
e_total_idxcount_3 = sum(e_total_idx(:) == 3);
e_total_idxcount_4 = sum(e_total_idx(:) == 4);
e_total1_idxcount_1 = sum(e_total1_idx(:) == 1);
e_total1_idxcount_2 = sum(e_total1_idx(:) == 2);
e_total1_idxcount_3 = sum(e_total1_idx(:) == 3);
e_total1_idxcount_4 = sum(e_total1_idx(:) == 4);
e_count = [e_total_idxcount_1,e_total_idxcount_2,e_total_idxcount_3,e_total_idxcount_4;
           e_total1_idxcount_1,e_total1_idxcount_2,e_total1_idxcount_3,e_total1_idxcount_4];

m_total_idxcount_1 = sum(m_total_idx(:) == 1);
m_total_idxcount_2 = sum(m_total_idx(:) == 2);
m_total_idxcount_3 = sum(m_total_idx(:) == 3);
m_total_idxcount_4 = sum(m_total_idx(:) == 4);
m_total1_idxcount_1 = sum(m_total1_idx(:) == 1);
m_total1_idxcount_2 = sum(m_total1_idx(:) == 2);
m_total1_idxcount_3 = sum(m_total1_idx(:) == 3);
m_total1_idxcount_4 = sum(m_total1_idx(:) == 4);
m_count = [m_total_idxcount_1,m_total_idxcount_2,m_total_idxcount_3,m_total_idxcount_4;
           m_total1_idxcount_1,m_total1_idxcount_2,m_total1_idxcount_3,m_total1_idxcount_4];

end

```

附录 Z t4_MBS_SBS

表格中 1 代表此时任务由微基站 1 接入，2 代表由微基站 2 接入，3 代表由微基站 3 接入，4 代表由宏基站接入。